



# Machine Learning

CMPE 257

Session 3: Advanced Supervised Learning Techniques

Instructor: Zara Hajihashemi, Ph.D.

# Recap

## Supervised Learning

- Logistic Regression
  - The values  $y$ , we now want to predict, take on only a small number of discrete values.
  - We choose the  $\theta$  values that **maximize this likelihood** — meaning the parameters that make the observed data most probable.

$$h_{\theta}(x) = g(\theta^T x) = \frac{1}{1 + e^{-\theta^T x}},$$

$$p(y^{(i)} | x^{(i)}; \theta) = (h_{\theta}(x^{(i)}))^{y^{(i)}} \cdot (1 - h_{\theta}(x^{(i)}))^{1-y^{(i)}}$$

- Generalized Linear Models: **maximum likelihood estimation under a chosen distribution.**
- Constructing GLMs:
  1. Pick a distribution for  $y$  from the exponential family.
  2. Combine inputs  $x$  linearly with parameters  $\theta$ .
  3. Use a link function to connect that linear combination to the mean of  $y$ .

# Agenda

- Decision Tree Techniques
- Motivation: Why decision trees?
- Partitioning the input space
- Splitting criteria (misclassification, cross-entropy, Gini)
- Regression trees and categorical features
- Transition to ensemble methods (bagging, random forests, boosting)

# Advanced Supervised Learning

## Decision Trees

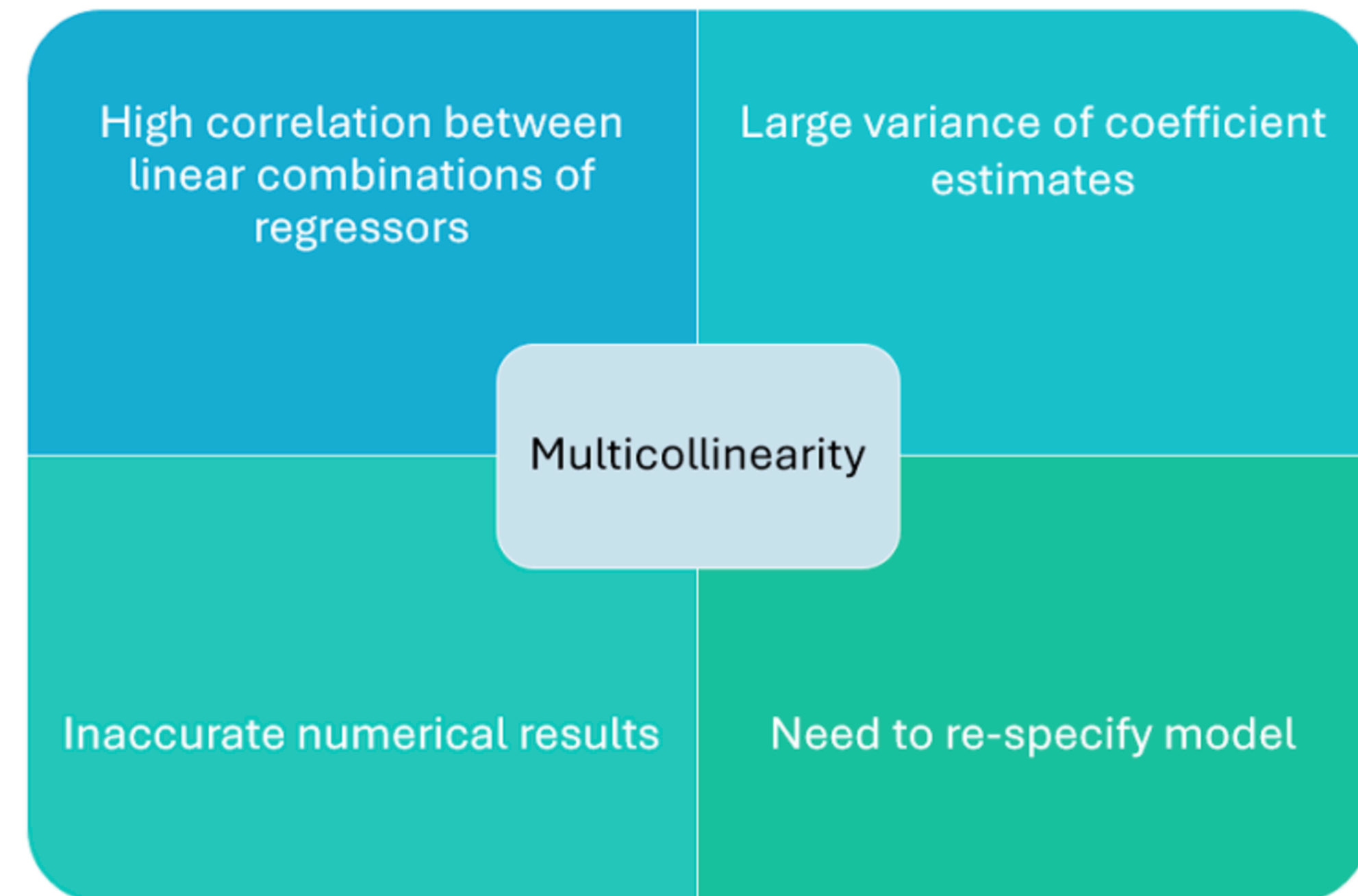
# Decision Trees

## Motivation

- The So far:
  - Linear Regression  $\rightarrow$  predicts continuous outcomes.
  - Logistic Regression / GLM  $\rightarrow$  predicts probabilities for classification.
- Limitations of linear regression:
  - Assumes a linear relationship
  - Sensitive to outliers:
    - outliers are data points that are far away from other observations.
    - The model's least-squares objective is to minimize the sum of squared errors, meaning a single outlier with a large error can significantly impact the fitted line and skew the results.

# Limitations of linear regression

- Issues with multicollinearity:
  - **Multicollinearity** happens when two or more pieces of data you feed into a model are so similar that they provide the exact same information.
  - Because the information is duplicated, the model gets confused trying to figure out which specific feature is actually driving the prediction.
  - Make it difficult to determine the individual effect of each independent variable on the dependent variable,
  - This results in unstable and unreliable coefficient estimates.



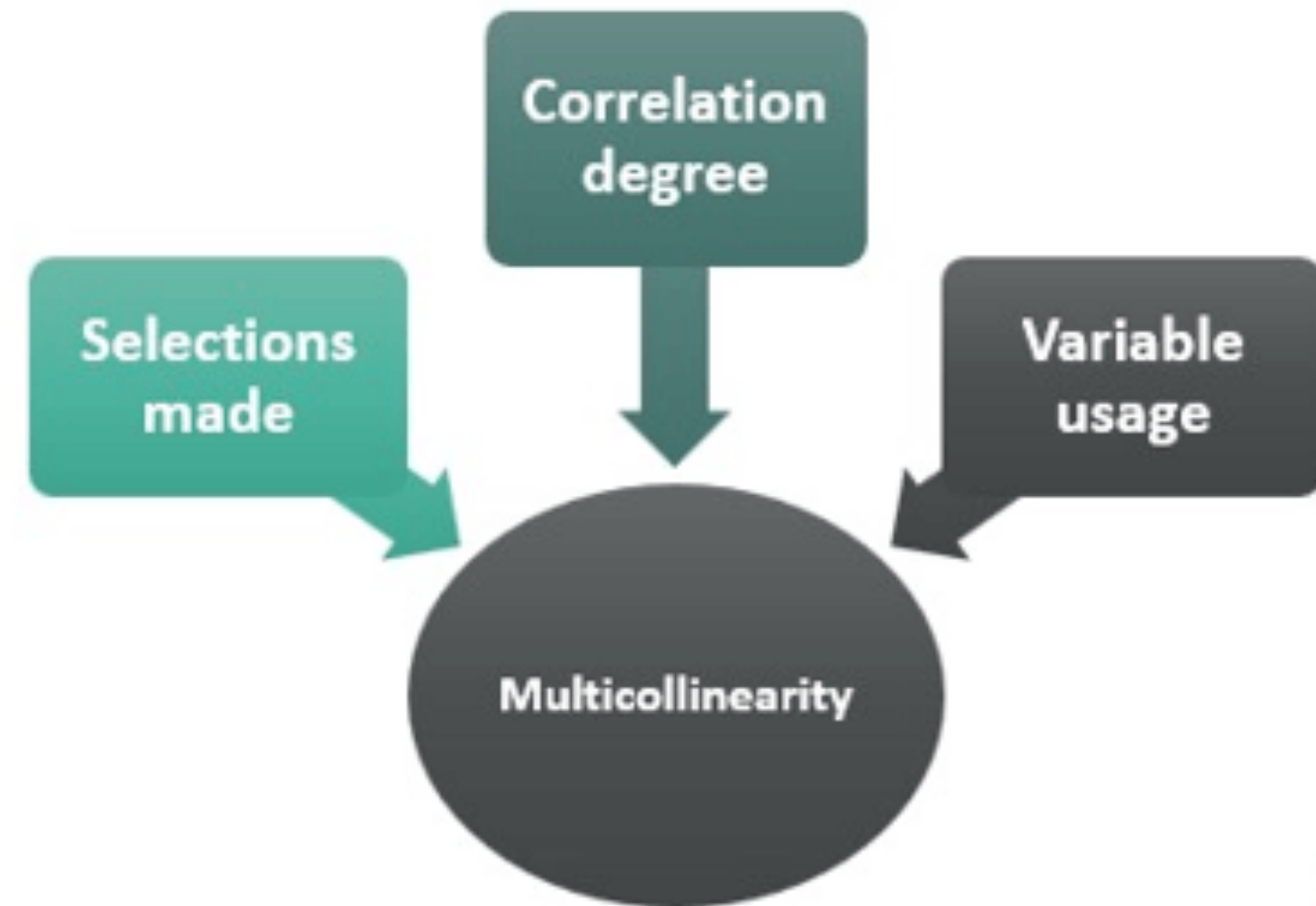
# Limitations of linear regression

Different kinds of multicollinearity:

**1. Perfect Multicollinearity:** This occurs when one predictor variable is an exact linear combination of one or more other predictors.

This causes serious problems because the regression model cannot distinguish the individual effects of these variables which makes it impossible to find their coefficients uniquely.

## Multicollinearity Causes



# Limitations of linear regression

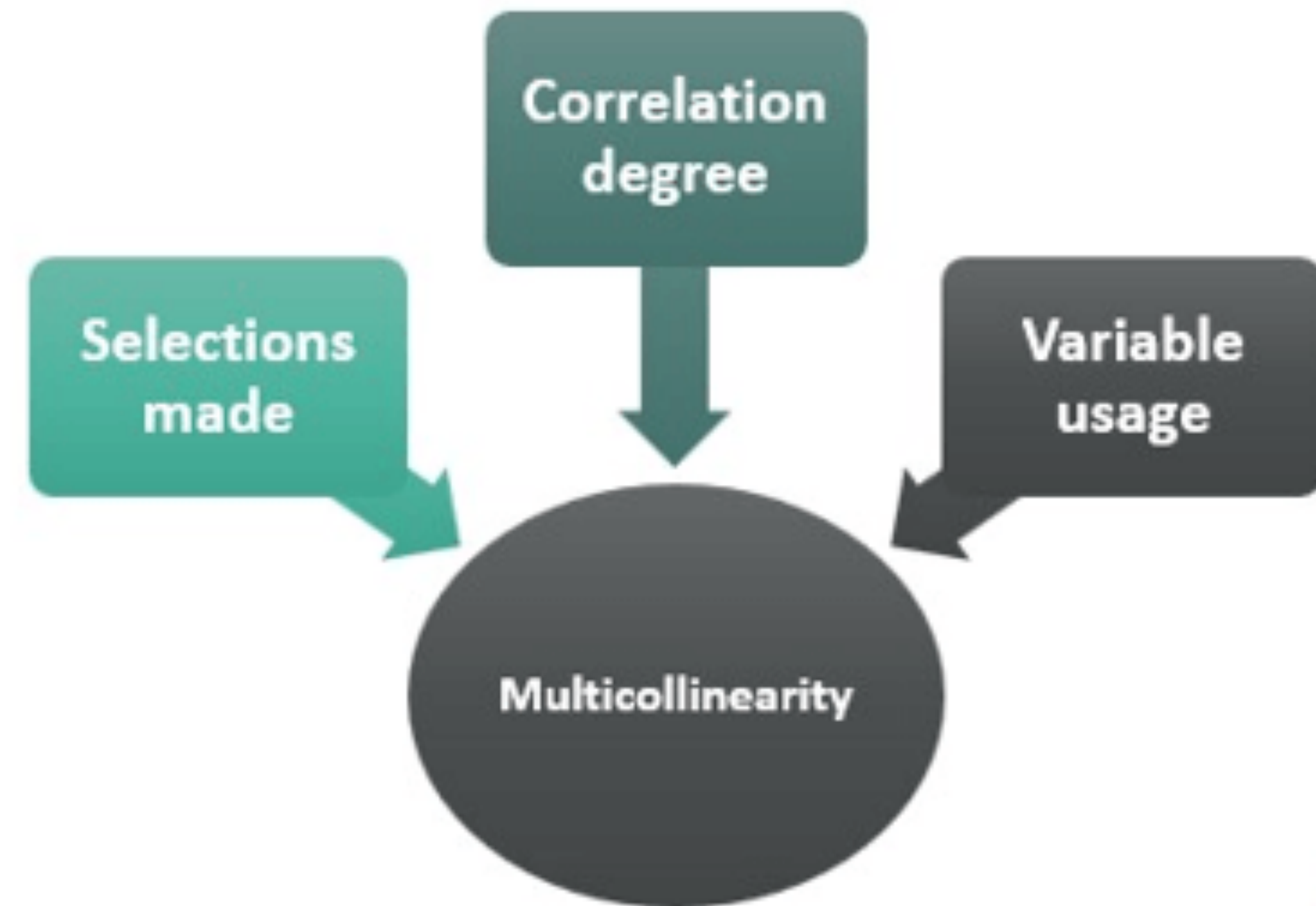
Different kinds of multicollinearity:

## 2. Imperfect (or Near) Multicollinearity:

This occurs when predictors are highly correlated but not perfectly.

For example height and weight have a strong positive correlation but are not exactly dependent. This can lead to unstable coefficient estimates where small changes in data cause large swings in the regression coefficients which makes the model less reliable.

Multicollinearity Causes



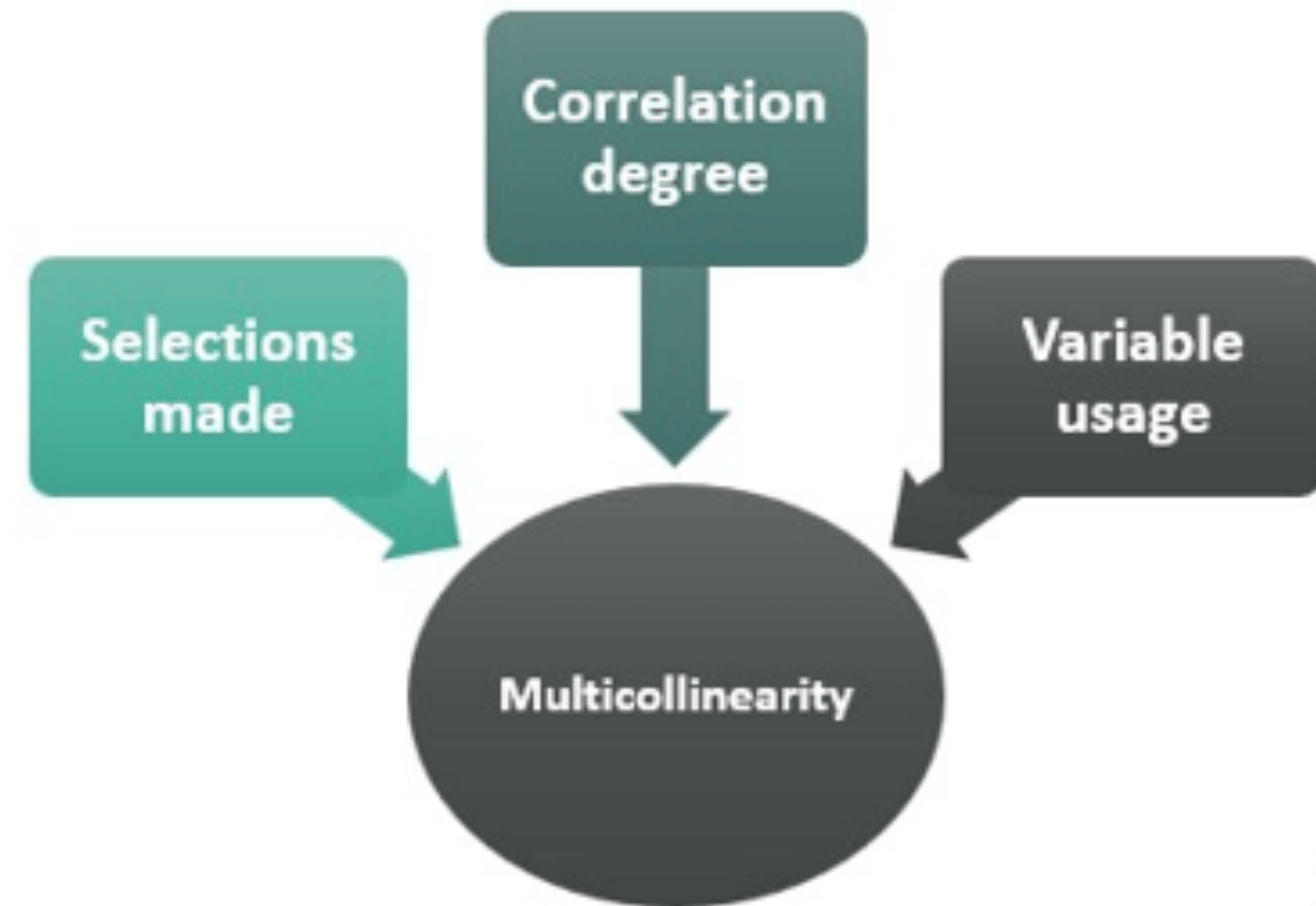
# Limitations of linear regression

Different kinds of multicollinearity:

**3. Structural Multicollinearity:** This type arises from how variables are created or defined. When new variables are formed by combining existing ones they become correlated.

For example if total income is calculated by adding salary, bonuses and investment returns these individual components will be strongly related to total income and to each other.

Multicollinearity Causes



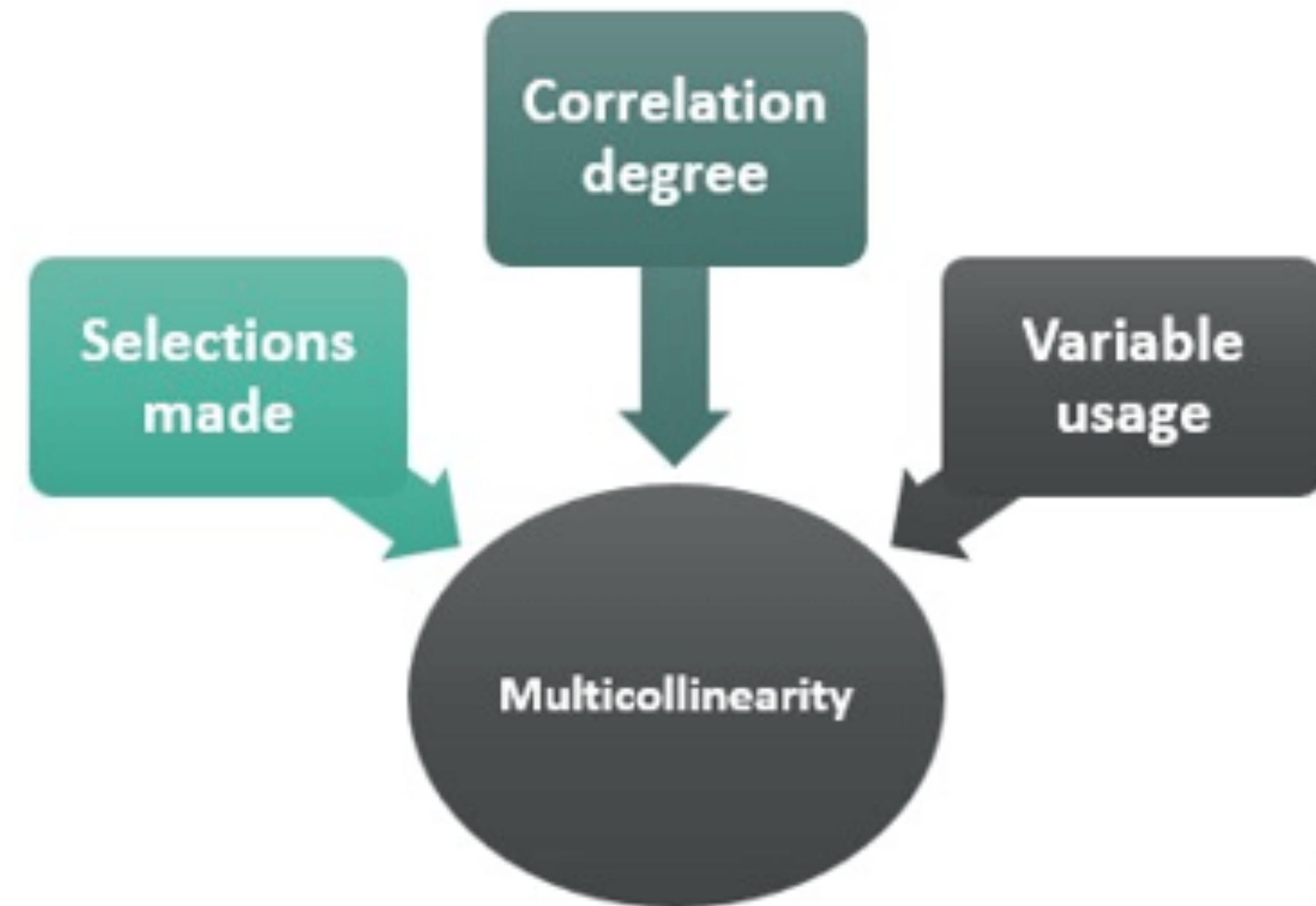
# Limitations of linear regression

Different kinds of multicollinearity:

**4. Data-based Multicollinearity:** This happens naturally due to the nature of the data or how it was collected, in observational studies where researchers have little control over variables.

For example years of education and age increase together which creates multicollinearity just because of the relationship in the population.

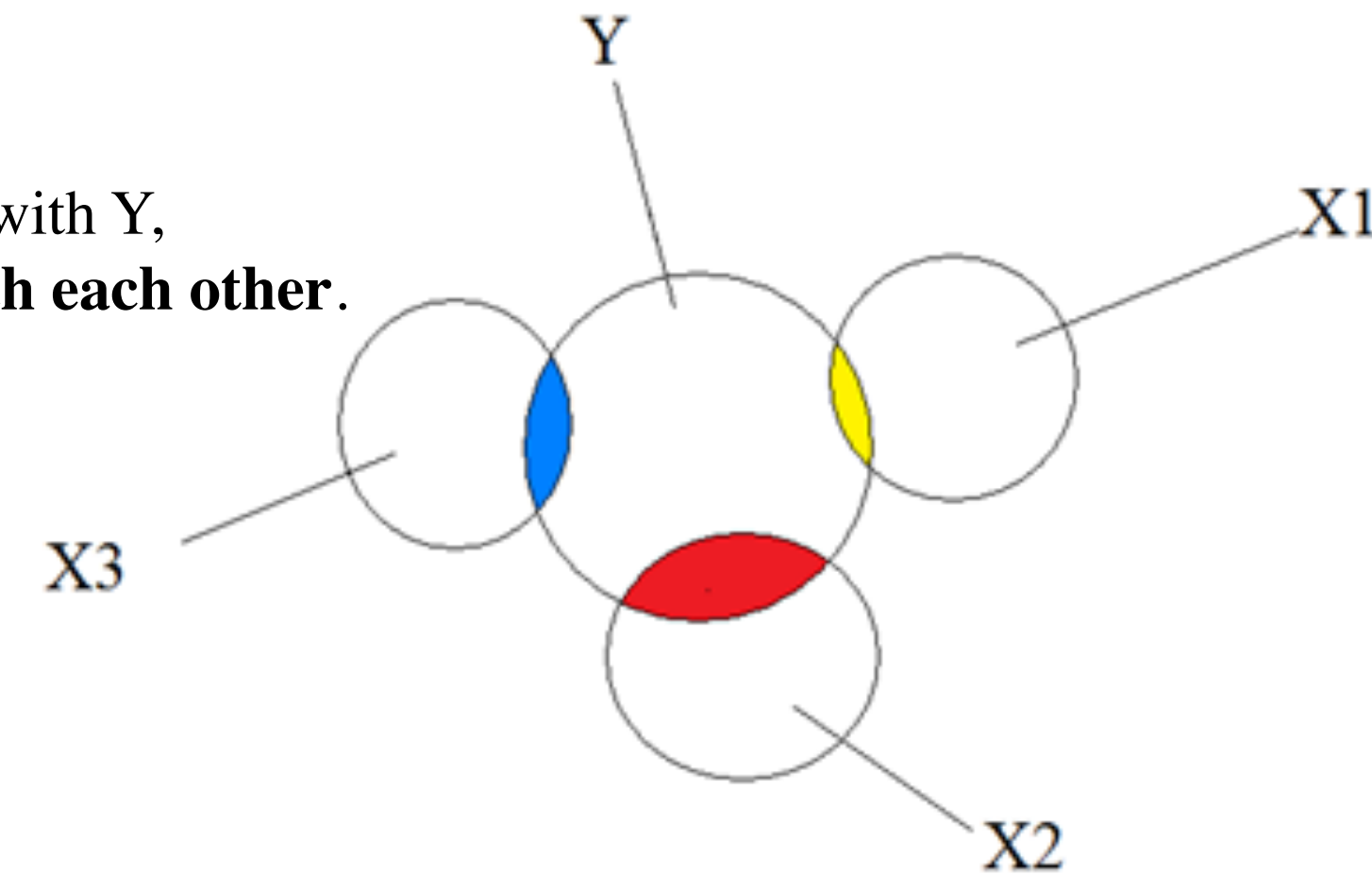
## Multicollinearity Causes



# Limitations of linear regression

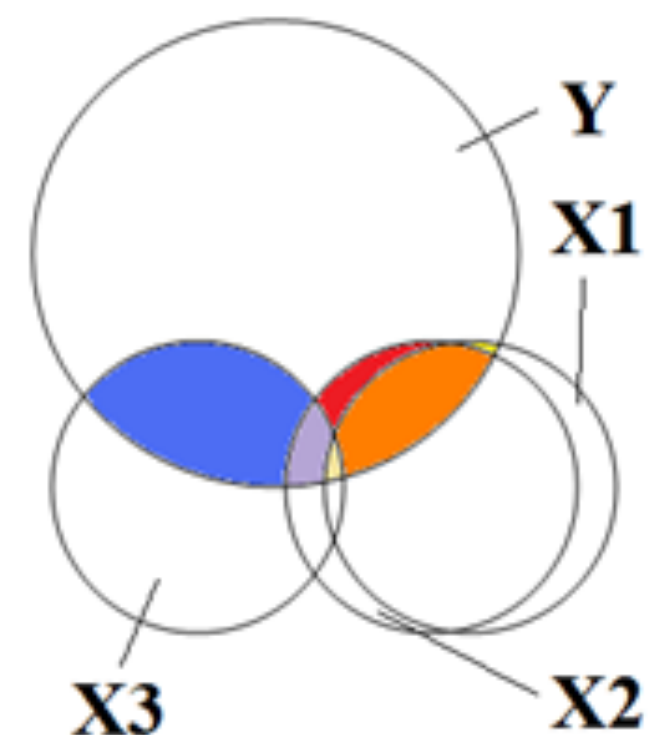
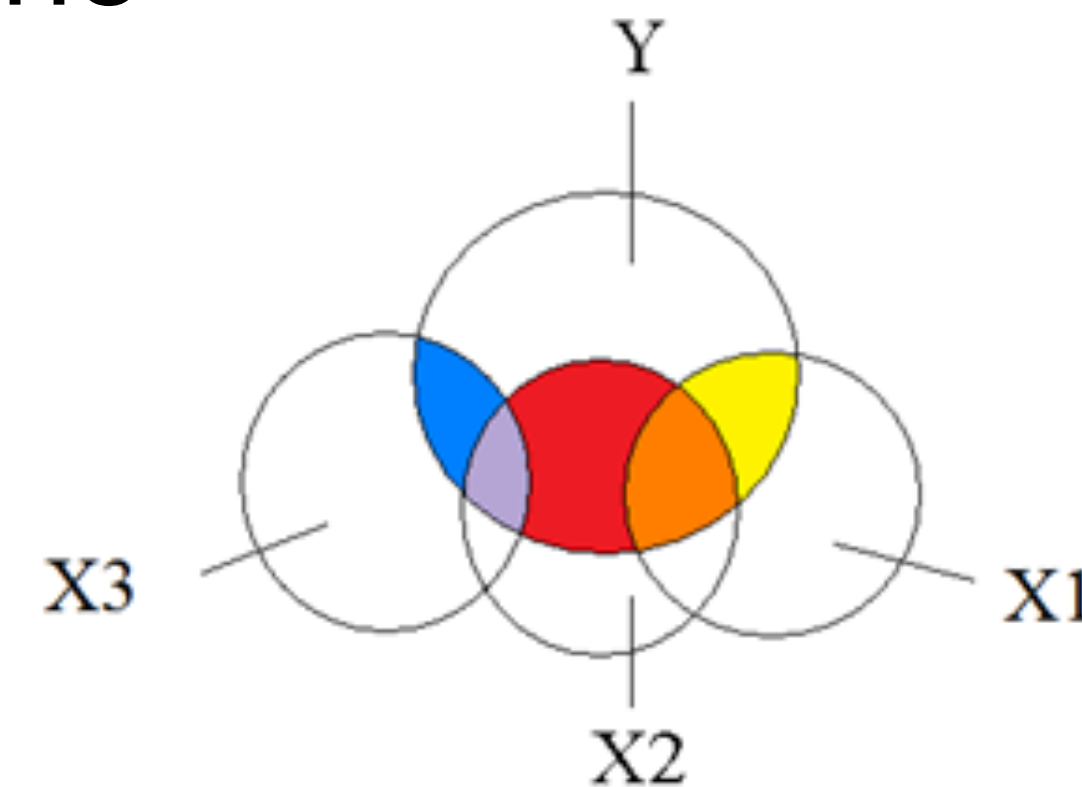
The Ideal Setup:

X1, X2, and X3 all overlap with Y,  
but **they do not overlap with each other.**



Problems of multicollinearity:

**1. Unstable Coefficients:** When independent variables are highly correlated, small changes in the data can cause large fluctuations in the regression coefficients. This instability makes the model's results unreliable.



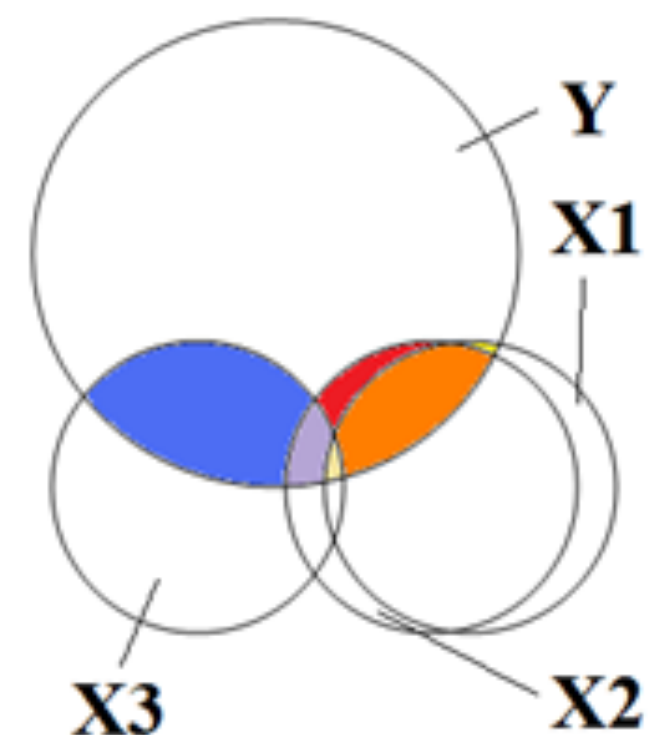
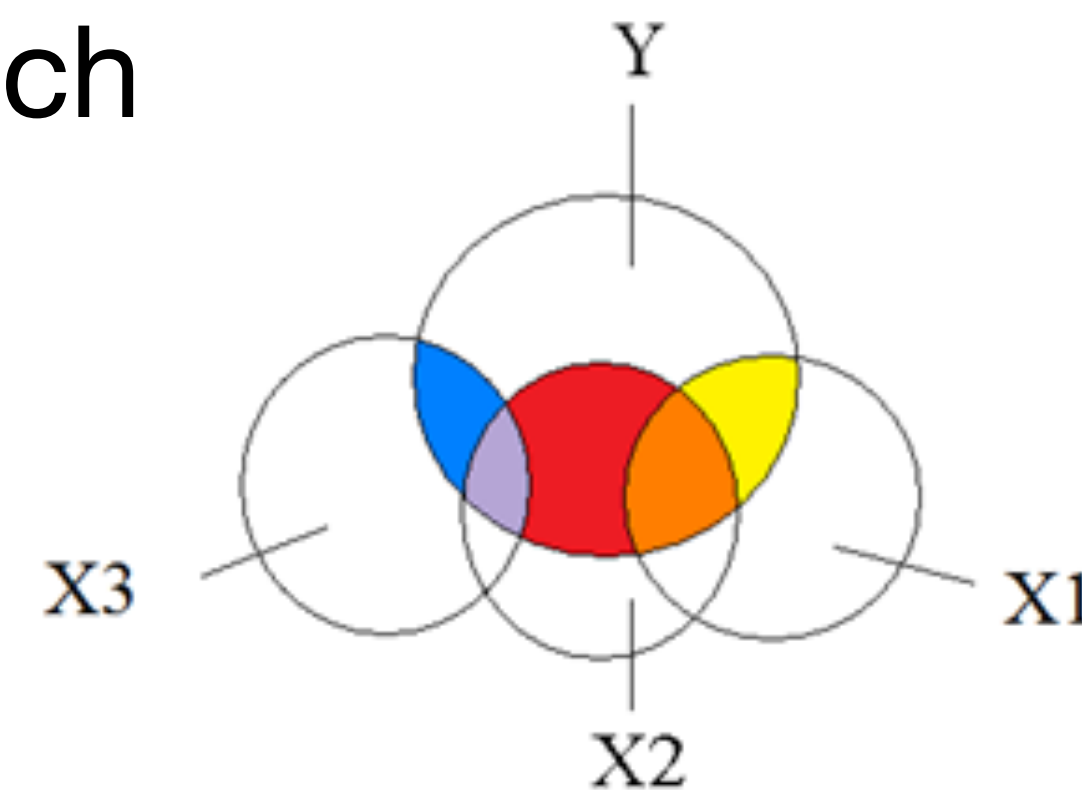
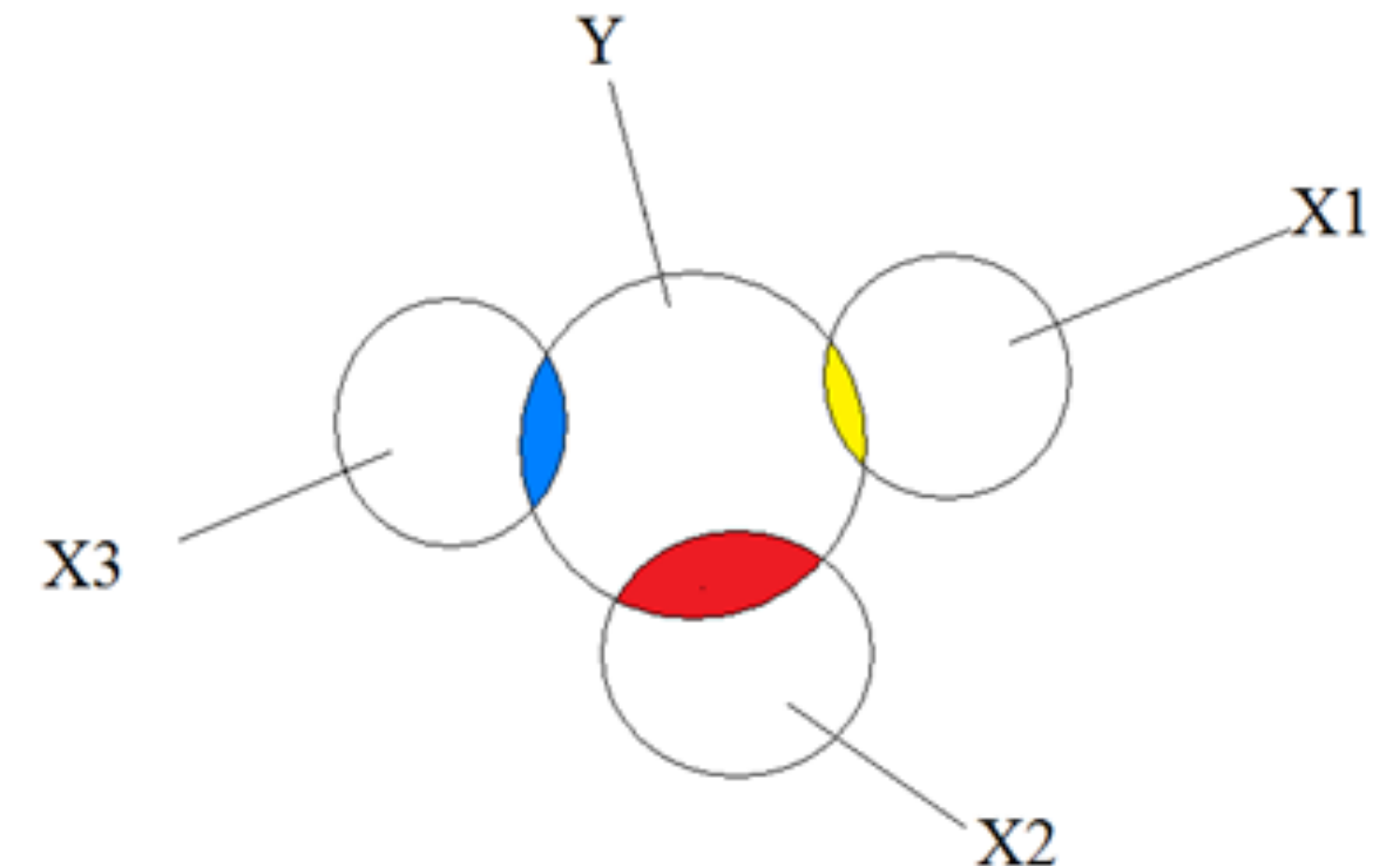
X1 and X2 provide the same information  
to the model

# Limitations of linear regression

Problems of multicollinearity:

**2. Reduced Interpretability:** It makes it difficult to understand how each predictor affects the dependent variable.

Since correlated variables are providing similar information, it's challenging to find the individual contribution of each predictor.

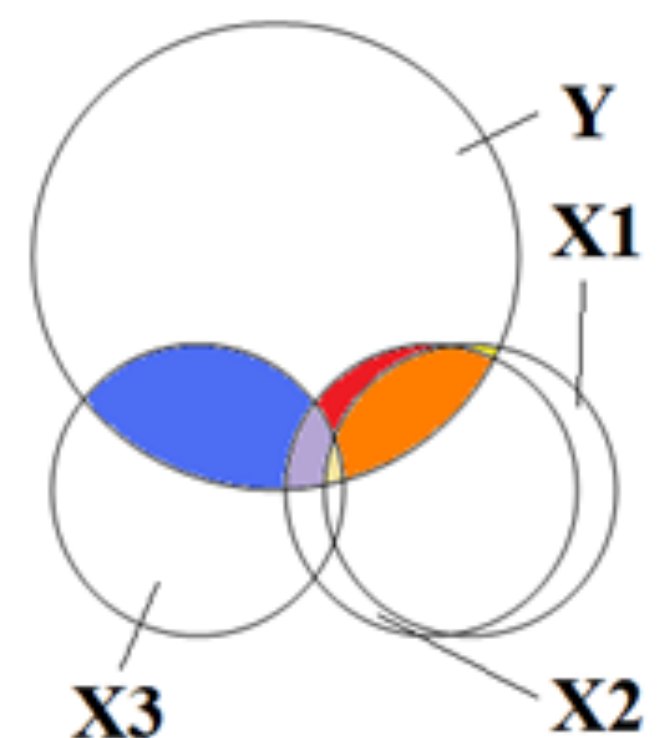
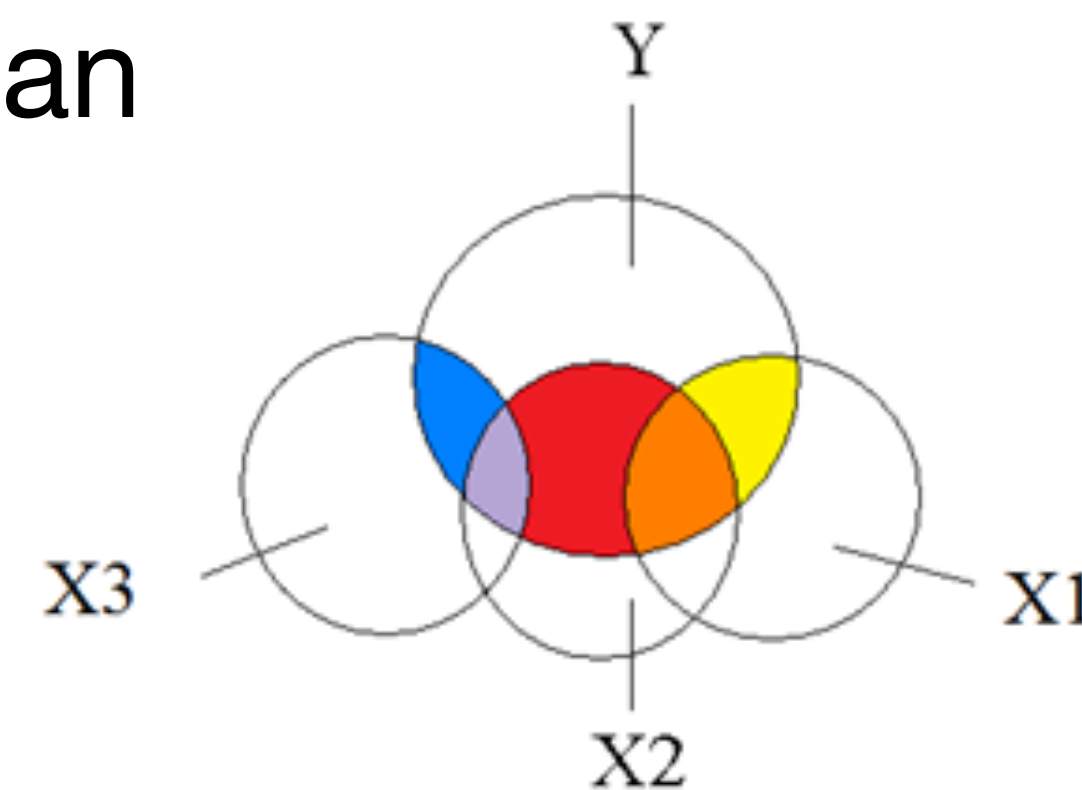
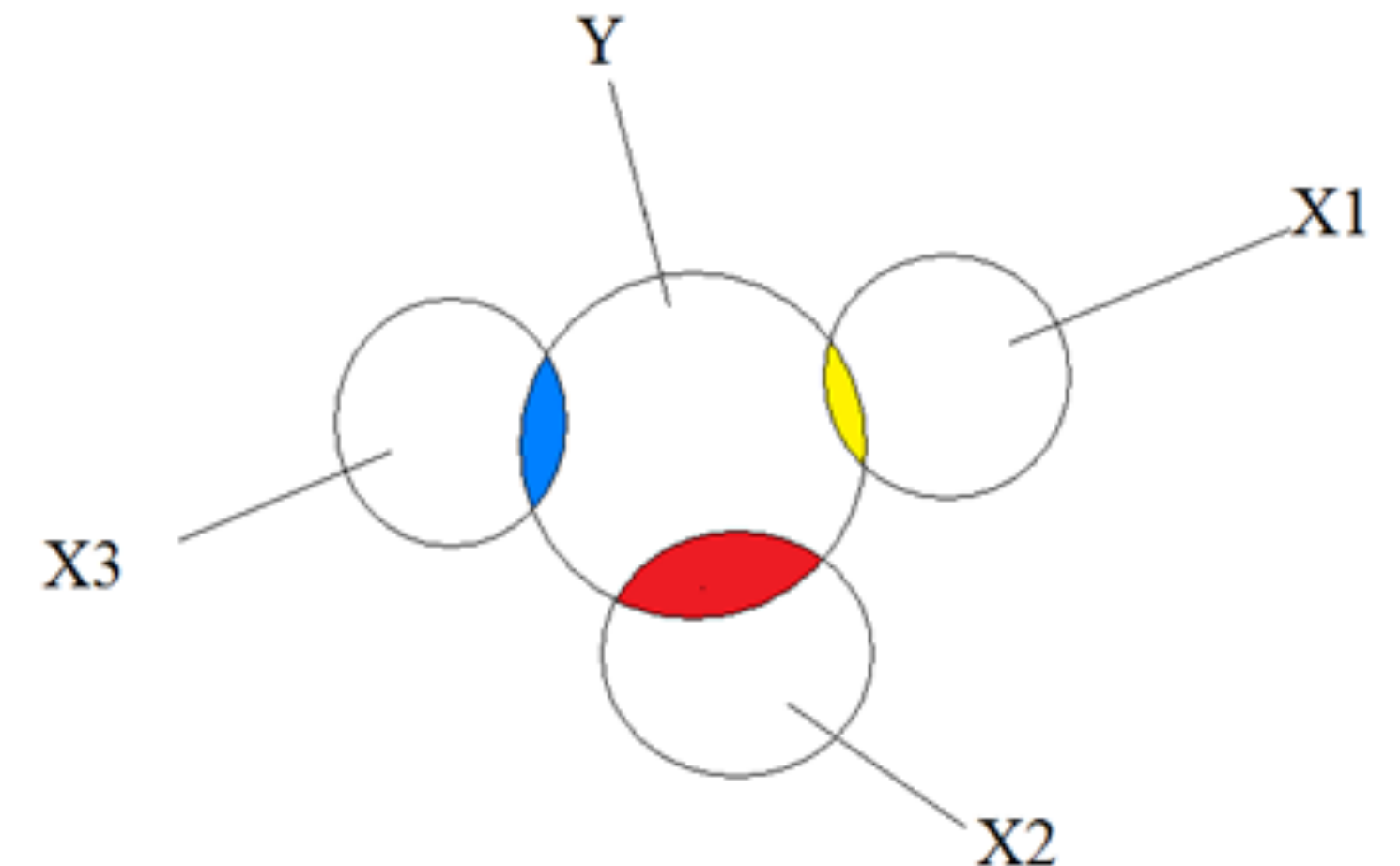


# Limitations of linear regression

Problems of multicollinearity:

**3. Less Reliable Predictions:** High multicollinearity can lead to standard errors which makes model's predictions less reliable.

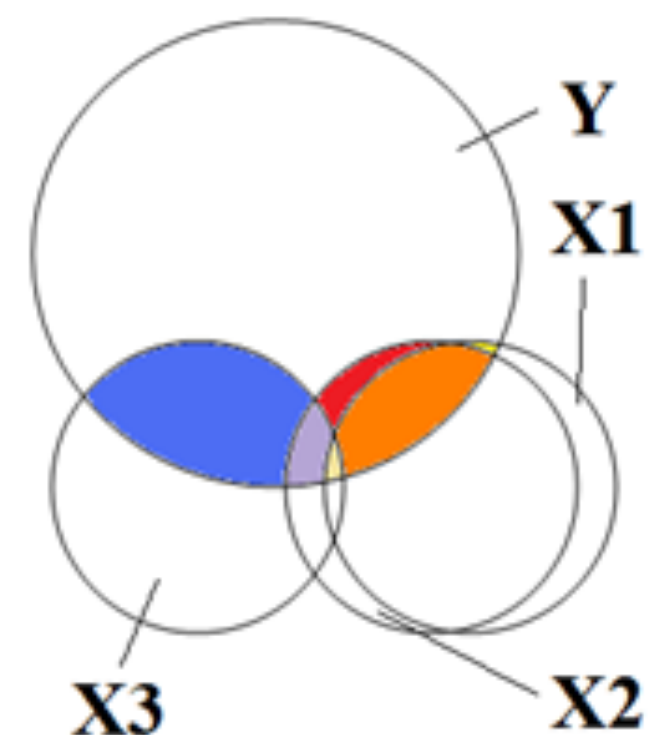
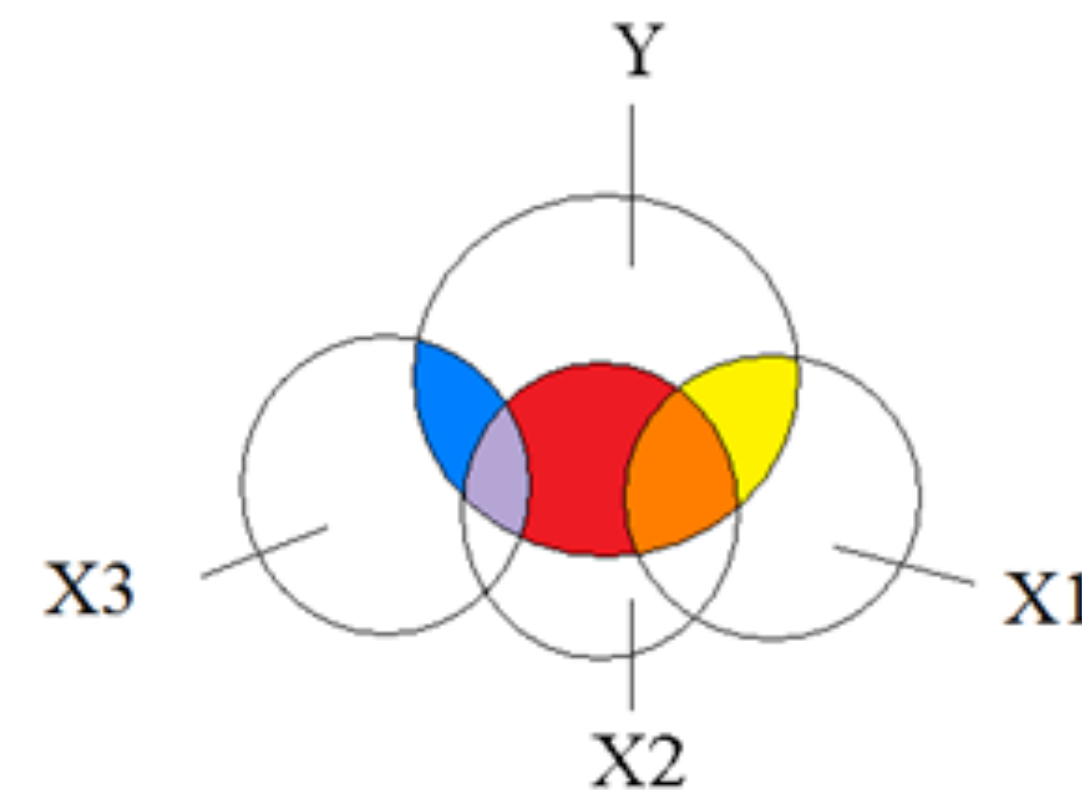
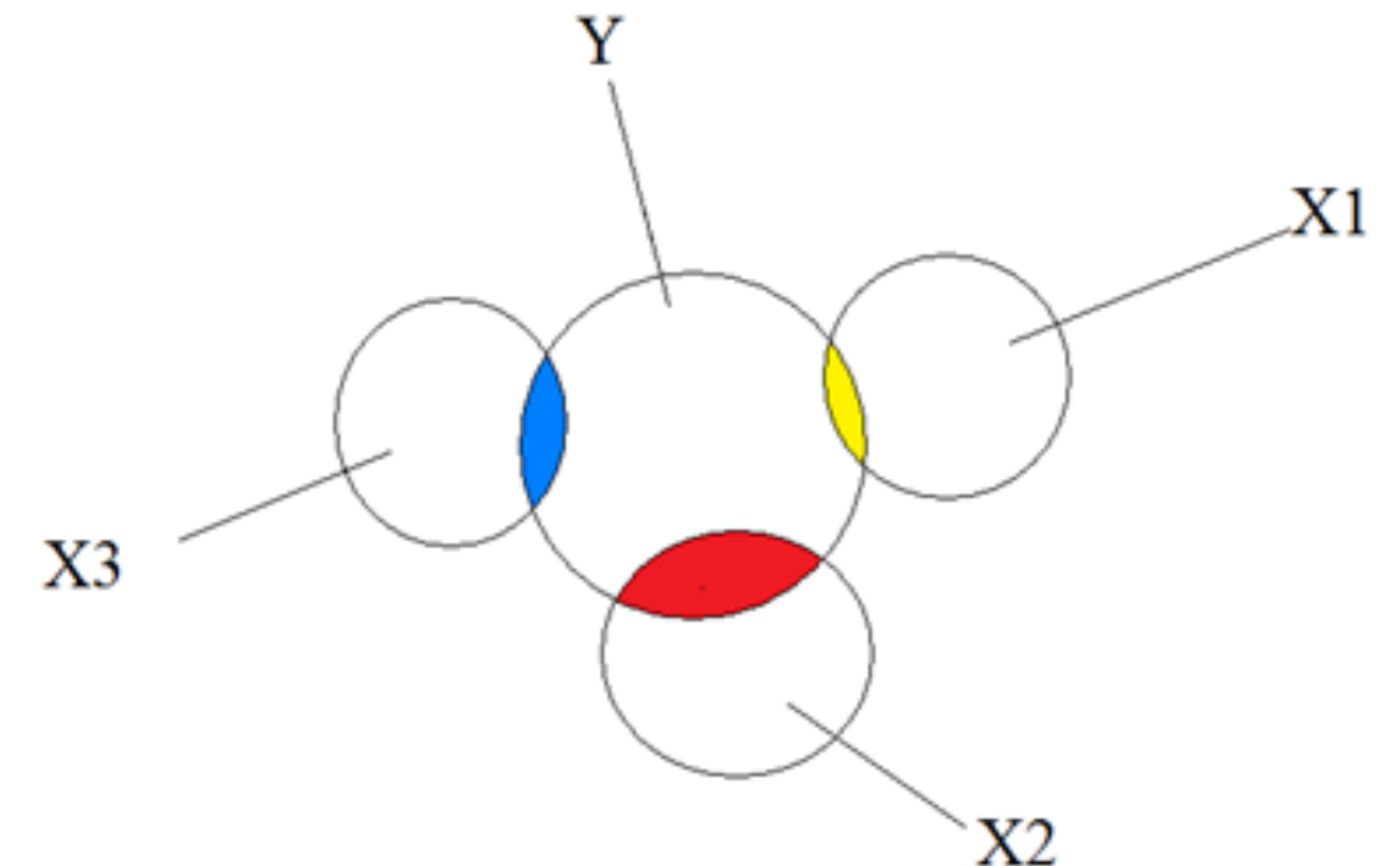
As a result, the model may not give results well to new unseen data and can give wrong results.



# Limitations of linear regression

Problems of multicollinearity:

**4. Risk of Overfitting:** The model may fit the training data too closely by capturing random noise which decrease its ability to work well on new data.



# Detecting Multicollinearity

1. Variance Inflation Factor (VIF)
2. Correlation Matrix
3. Pairwise Scatter Plots

# Detecting Multicollinearity

## 1. Variance Inflation Factor (VIF)

VIF is used to detect and measure multicollinearity in multiple linear regression.

It shows how much the variance of a regression coefficient increases due to correlation with other predictor variables.

$$VIF_i = \frac{1}{1 - R_i^2}$$


**Coefficient of Determination** of variable ranges from 0 to 1.

It measures what percentage of that specific feature's variance can be explained by the rest of the dataset.

- **VIF  $\approx$  1:**  $R^2 = 0$ ; No correlation with other predictors
- **$1 < \text{VIF} \leq 5$ :** Mild to moderate correlation (usually fine)
- **VIF  $> 10$  or higher:**  $R^2$  close to 1, the feature is almost perfectly redundant therefore Strong multicollinearity

# Detecting Multicollinearity

## 2. Covariance Matrix

Shows the correlation coefficients between every pair of variables.

**Correlation matrix** is a table that shows how different variables are related to each other.

High absolute values (close to 1 or -1) shows strong relationships.

Such pairs are likely sources of multicollinearity.

Dataframe is :

|   | x  | y  | z  |
|---|----|----|----|
| 0 | 45 | 38 | 10 |
| 1 | 37 | 31 | 15 |
| 2 | 42 | 26 | 17 |
| 3 | 35 | 28 | 21 |
| 4 | 39 | 33 | 12 |

Correlation matrix is :

|   | x         | y         | z         |
|---|-----------|-----------|-----------|
| x | 1.000000  | 0.518457  | -0.701886 |
| y | 0.518457  | 1.000000  | -0.860941 |
| z | -0.701886 | -0.860941 | 1.000000  |

# Detecting Multicollinearity

## 2. Covariance Matrix

- **No Correlation:** A correlation value of 0 means no linear relationship between the variables. As one changes, the other does not follow any predictable pattern.
- **Positive Correlation:** A value closer to +1 indicates a direct relationship as one variable increases, the other also increases. Example: height and weight.
- **Negative Correlation:** A value closer to -1 indicates an inverse relationship as one variable increases, the other decreases. Example: speed and travel time.

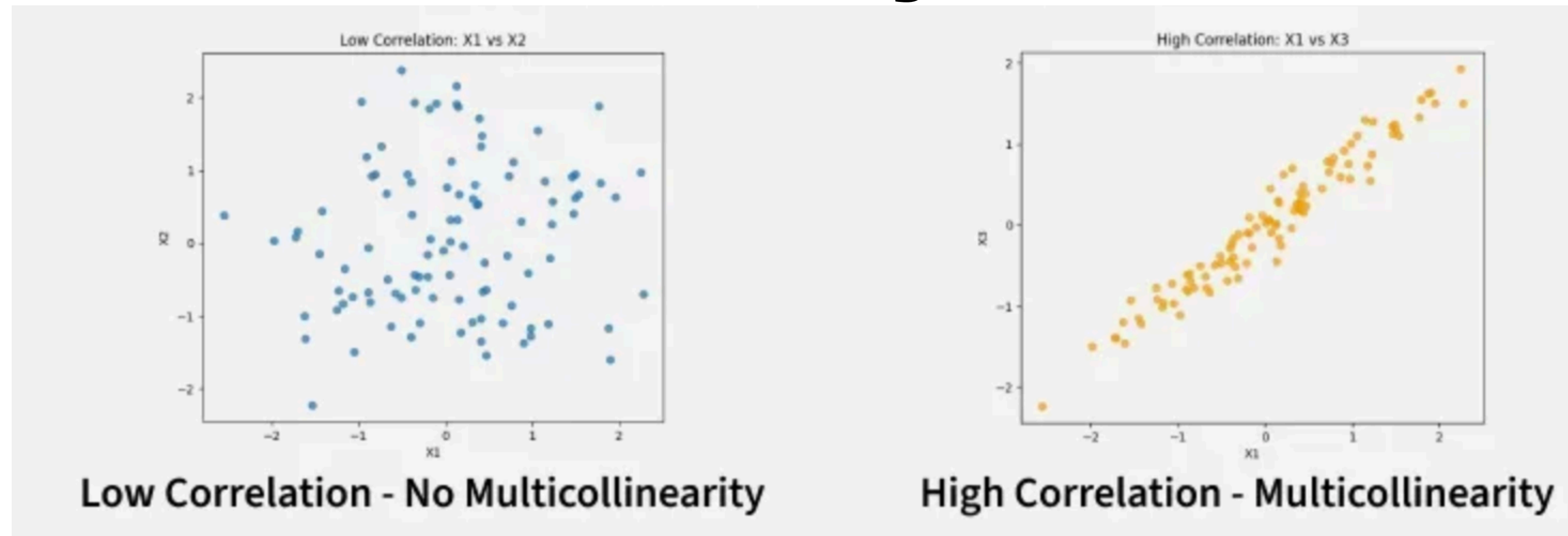
Dataframe is :

|   | x  | y  | z  |
|---|----|----|----|
| 0 | 45 | 38 | 10 |
| 1 | 37 | 31 | 15 |
| 2 | 42 | 26 | 17 |
| 3 | 35 | 28 | 21 |
| 4 | 39 | 33 | 12 |

Correlation matrix is :

|   | x         | y         | z         |
|---|-----------|-----------|-----------|
| x | 1.000000  | 0.518457  | -0.701886 |
| y | 0.518457  | 1.000000  | -0.860941 |
| z | -0.701886 | -0.860941 | 1.000000  |

# Detecting Multicollinearity



## 3. Pairwise Scatter Plots

Plotting variables against each other lets us visually check their relationships.

Strong linear patterns suggest high correlation.

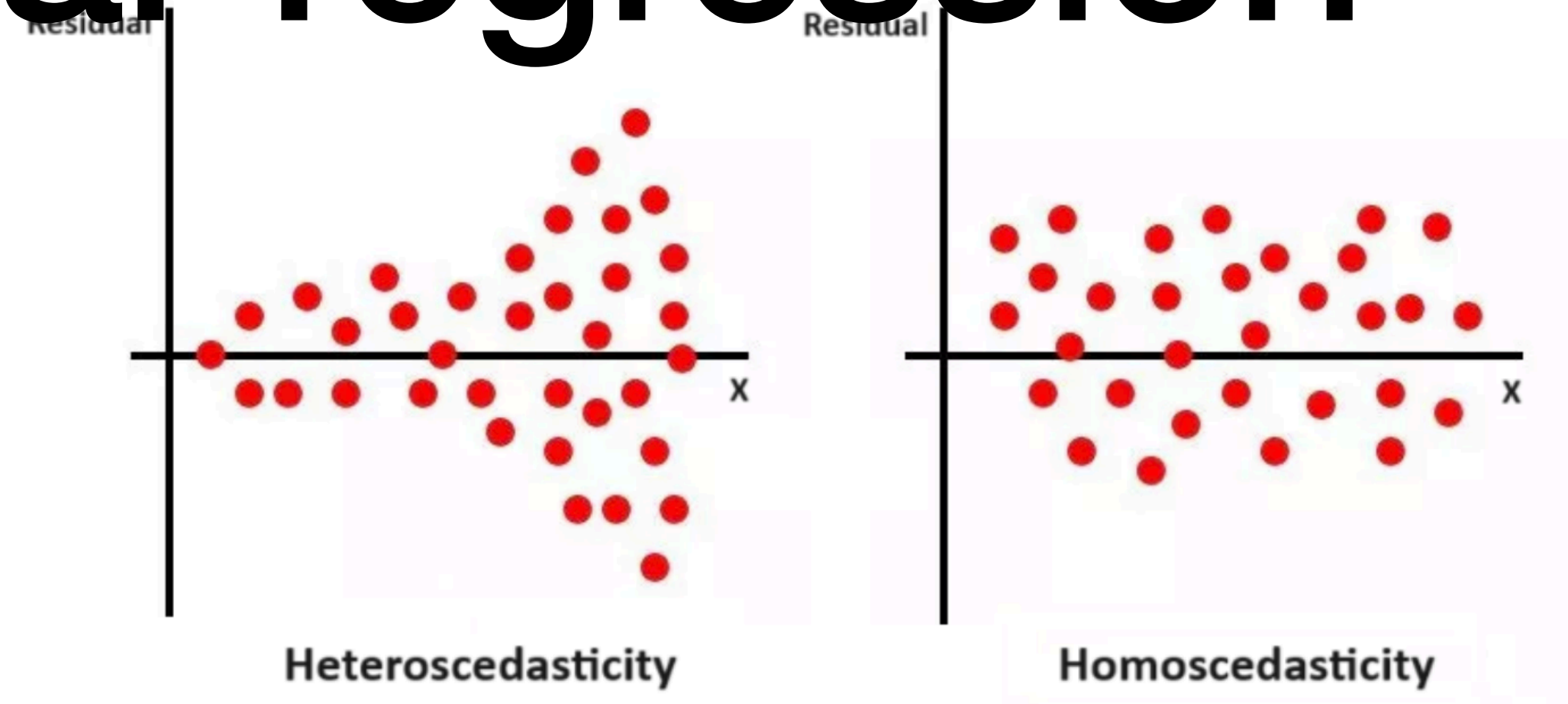
Although this method takes time with many variables.

# Limitations of linear regression

- Limitations of linear regression:

- **Homoscedasticity assumption:**

- Definition: the variance of the error terms (residuals) is constant across all levels of the independent variables.
- In other words, the variance of the residual, or error term, in a regression model is constant across all levels of the independent variable. This means the spread of data points around the regression line is roughly the same, regardless of the value of the predictor variable.
- If this assumption is violated (a condition called **heteroscedasticity**), the standard errors of the coefficients can be inaccurate.
- This affects the statistical significance of the model's parameters.



# Limitations of linear regression

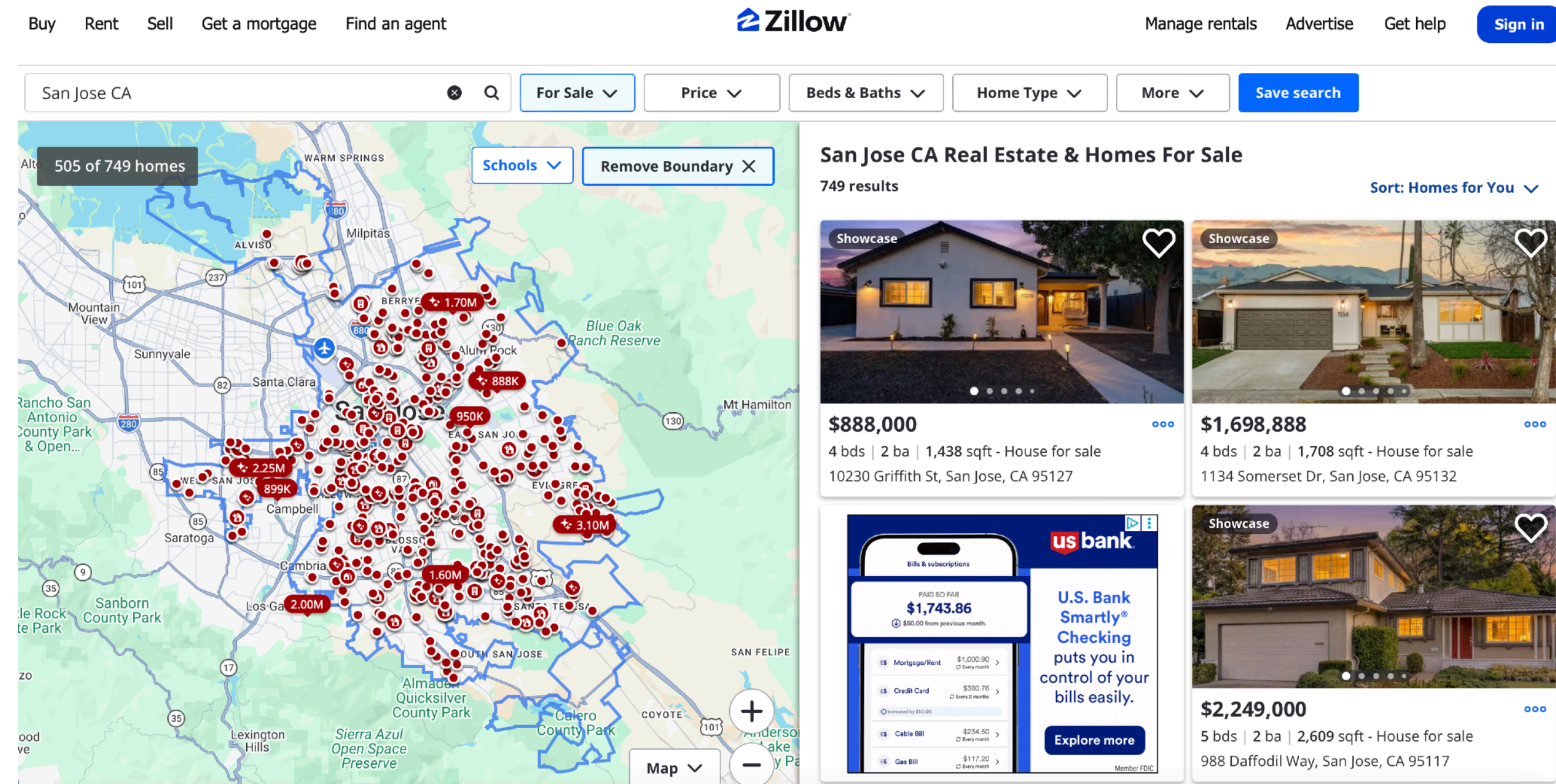
## Example

- House Price Model

Imagine you are building a model to predict House Price using:

1. Square Footage (Total area)
2. Number of Rooms

Multicollinearity?



# Limitations of linear regression

## Example

- Monthly Grocery Bills Model

Imagine you are predicting monthly food spend based on annual income.

**Homoscedasticity?**



# Limitations of linear regression

## Example

- Monthly Grocery Bills Model

Imagine you are predicting monthly food spend based on annual income.

**Homoscedasticity:** Whether a person earns \$30,000 or \$300,000, your prediction error is always roughly +/- \$50. The "spread" of your mistakes is consistent.

**The Reality (Heteroscedasticity):** In the real world, this usually fails. People with low incomes have very little flexibility; they all spend a similar, small amount on food. However, people with very high incomes have huge variety—some eat cheaply, while others spend thousands on fine dining and organic imports.



# Limitations of linear regression

- Limitations of logistic regression:
  - Assumes a linear relationship with log-odds.
  - **Odds:** This is the ratio of the probability that an event will happen to the probability that it won't.
    - For example, if the probability of buying a car is 0.75, the odds are  $0.75 / (1 - 0.75) = 3$ . This means it's 3 times more likely that someone will buy a car than not buy one.
  - **Log-odds:** This is simply the logarithm of the odds. The log-odds can range from negative infinity to positive infinity, which makes it easier for a linear model to work with.
  - Logistic regression assumes that if you increase a person's income by, say, \$10,000, their log-odds of buying a car will increase by a fixed, constant amount.

# Limitations of logistic regression

- Limitations of logistic regression:
  - Assumes a linear relationship with log-odds.
    - **Linear relationship:** This means that as an independent variable (like income) changes by a certain amount, the log-odds of the outcome (buying a car) change by a consistent, predictable amount.
    - ***Relies on linearity for decision boundary:*** logistic regression creates a linear decision boundary to separate classes.
    - This means it performs poorly when the data is not linearly separable and the relationship between classes is complex or non-linear.

# Limitations of logistic regression

- Limitations of logistic regression:
  - Assumes a linear relationship with log-odds
  - **Can overfit with high-dimensional data:** For datasets with a high number of features relative to the number of observations, logistic regression is susceptible to overfitting.

# Limitations of logistic regression

- Limitations of logistic regression:
  - Assumes a linear relationship with log-odds
  - Can overfit with high-dimensional data
  - **Doesn't capture complex/hierarchical relationships automatically**
  - Sensitive to outliers
  - Requires sufficient data

# Limitations of logistic regression

- Limitations of logistic regression:
  - Assumes a linear relationship with log-odds
  - Can overfit with high-dimensional data
  - Doesn't capture complex relationships automatically
  - **Sensitive to outliers:** can disproportionately affect the model's coefficients and decision boundary.
  - Requires sufficient data

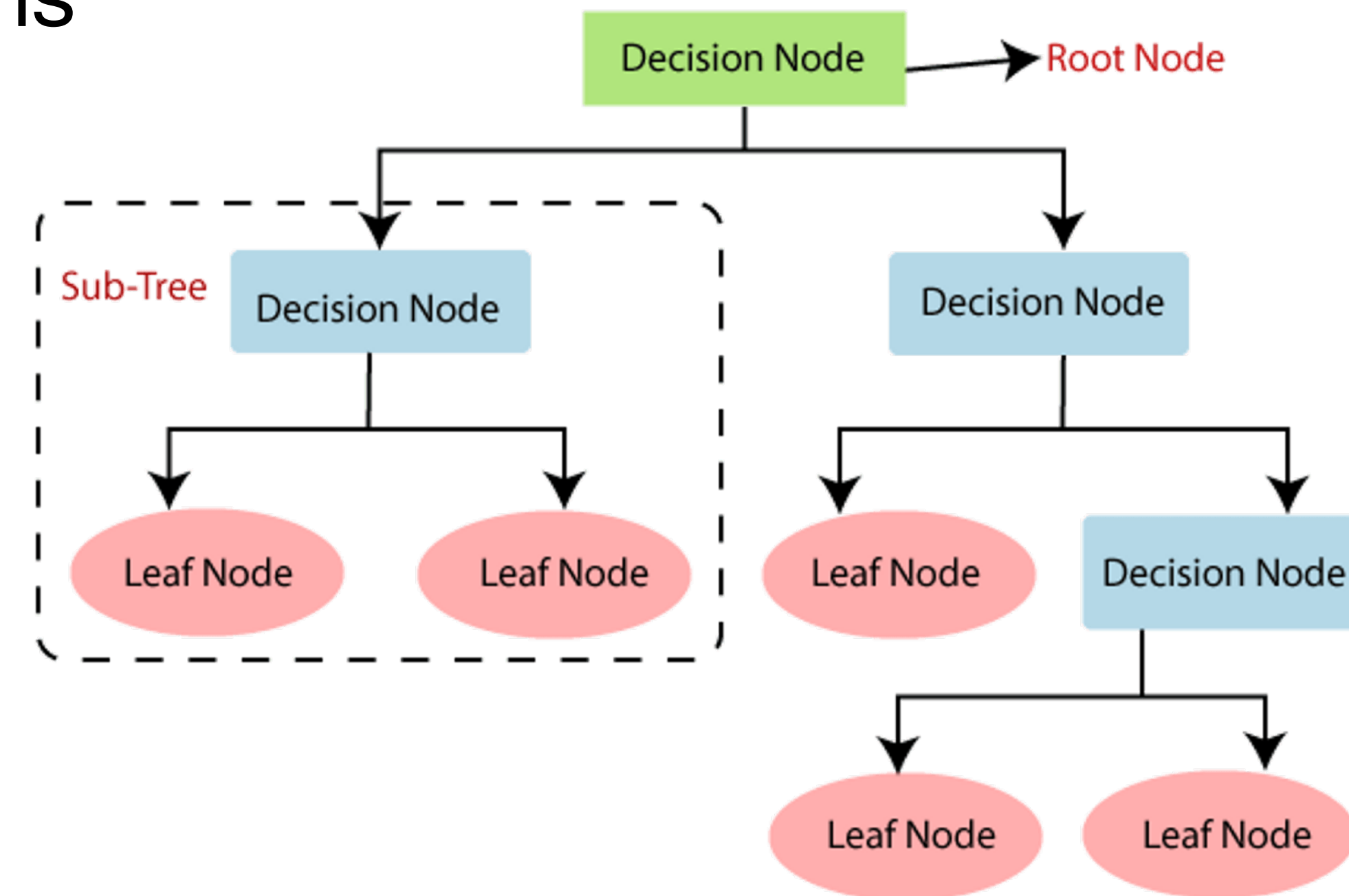
# Limitations of logistic regression

- Limitations of logistic regression:
  - Assumes a linear relationship with log-odds
  - Can overfit with high-dimensional data
  - Doesn't capture complex relationships automatically
  - Sensitive to outliers
- **Requires sufficient data:** Logistic regression needs a sufficient number of observations for the model to be reliable. If the sample size is small, especially when predicting a rare event, the model can suffer from overfitting and unstable coefficient estimates.

# Decision Trees

## Intuition

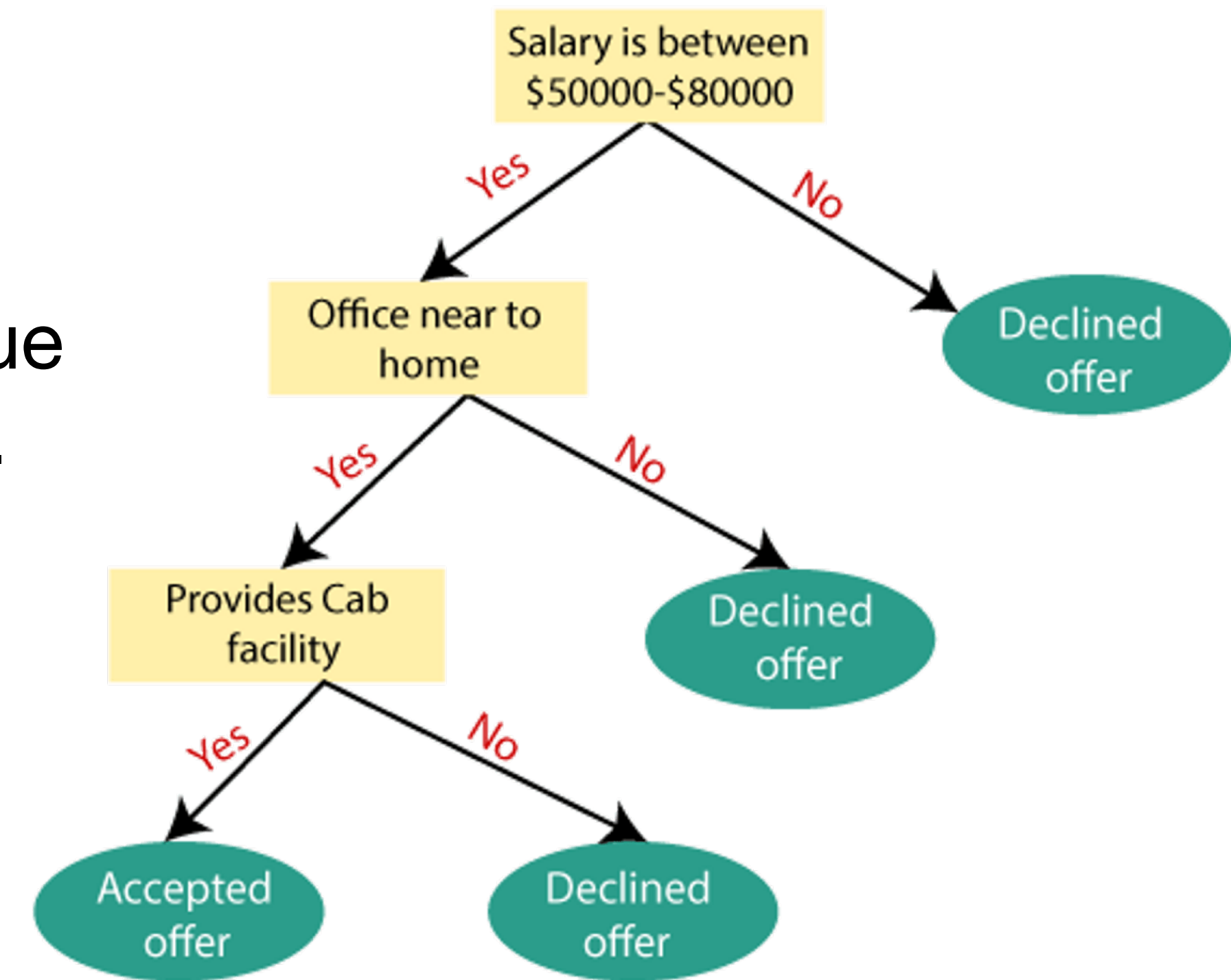
- Decision Trees are supervised learning algorithms used for classification and regression problems.
- Partition the space into regions
  - Divide the feature space step by step into smaller, more homogeneous regions.
  - Each region corresponds to a “leaf” of the tree.



# Decision Trees

## Intuition

- They work by creating a model that predicts the value of a target variable based on several input variables.
- The model is a tree-like structure:
  - Each internal **node** representing a "**test**" on an attribute,
  - Each **branch** representing the **outcome** of the test, and each leaf node representing a class label.
- **Key idea:** Instead of fitting one equation to all the data, we **break the space into pieces** and fit simple predictions inside each piece.



# Decision Trees

## How to choose split?

- Choose split that reduces loss the most  $\mathcal{L}(\mathcal{R})$
- $\mathcal{L}(\mathcal{R})$ : loss of split on region  $\mathcal{R}$
- For now lets assume our loss is the Miss classification loss
  - How many examples in your region you get wrong

$\hat{p}_c$ : given C classes total, you can define  $\hat{p}_c$  to be the proportion of examples in  $\mathcal{R}$  that are of class  $c$ .

$$\mathcal{L}_{\text{misclass}} = 1 - \max_c(\hat{p}_c)$$


Loss/error of misclassification

# Decision Trees

## Formal Definition

- We are looking for a ***split function*** to define ***regions***.

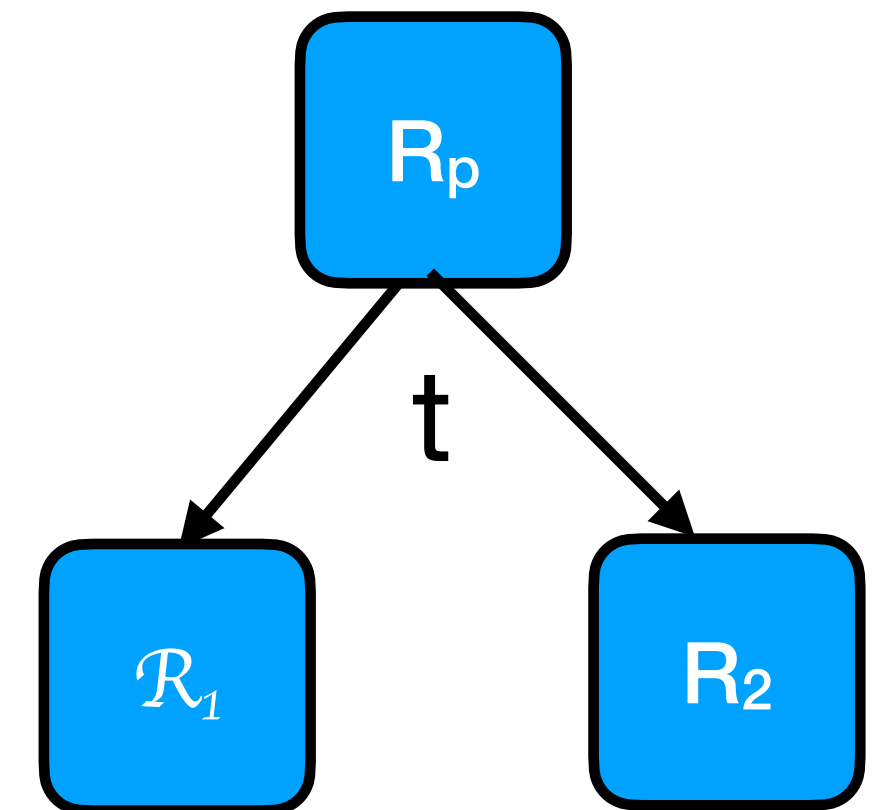
$$s_p(j, t) = (\{X \mid X_j < t, X \in \mathcal{R}_p\},$$

$$\{X \mid X_j \geq t, X \in \mathcal{R}_p\})$$

$t$ : threshold

$j$ : feature index

$\mathcal{R}_p$ : parent region



# Decision Trees

## Formal Definition

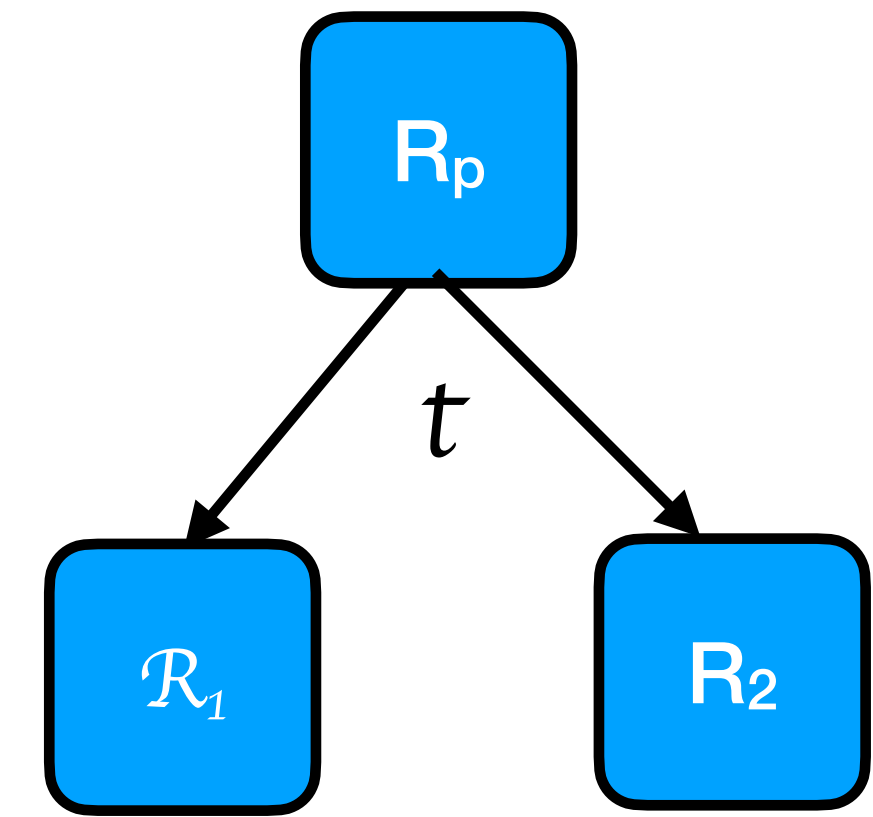
- The goal is to reduce the loss as much as possible

$$\min_{j,t} L(R_p) - (L(R_1) + L(R_2))$$

The  $\mathcal{L}(\mathcal{R}_p)$  = loss of the **parent region**

$\mathcal{L}(\mathcal{R}_1), \mathcal{L}(\mathcal{R}_2)$  = losses of the **child regions** after splitting on feature  $j$  at threshold  $t$ .

This is the **split criterion** for decision trees: find the feature  $j$  and threshold  $t$  that minimize the children's loss relative to the parent's loss (i.e., maximize the gain).



$$s_p(j,t) = (\{X \mid X_j < t, X \in \mathcal{R}_p\},$$

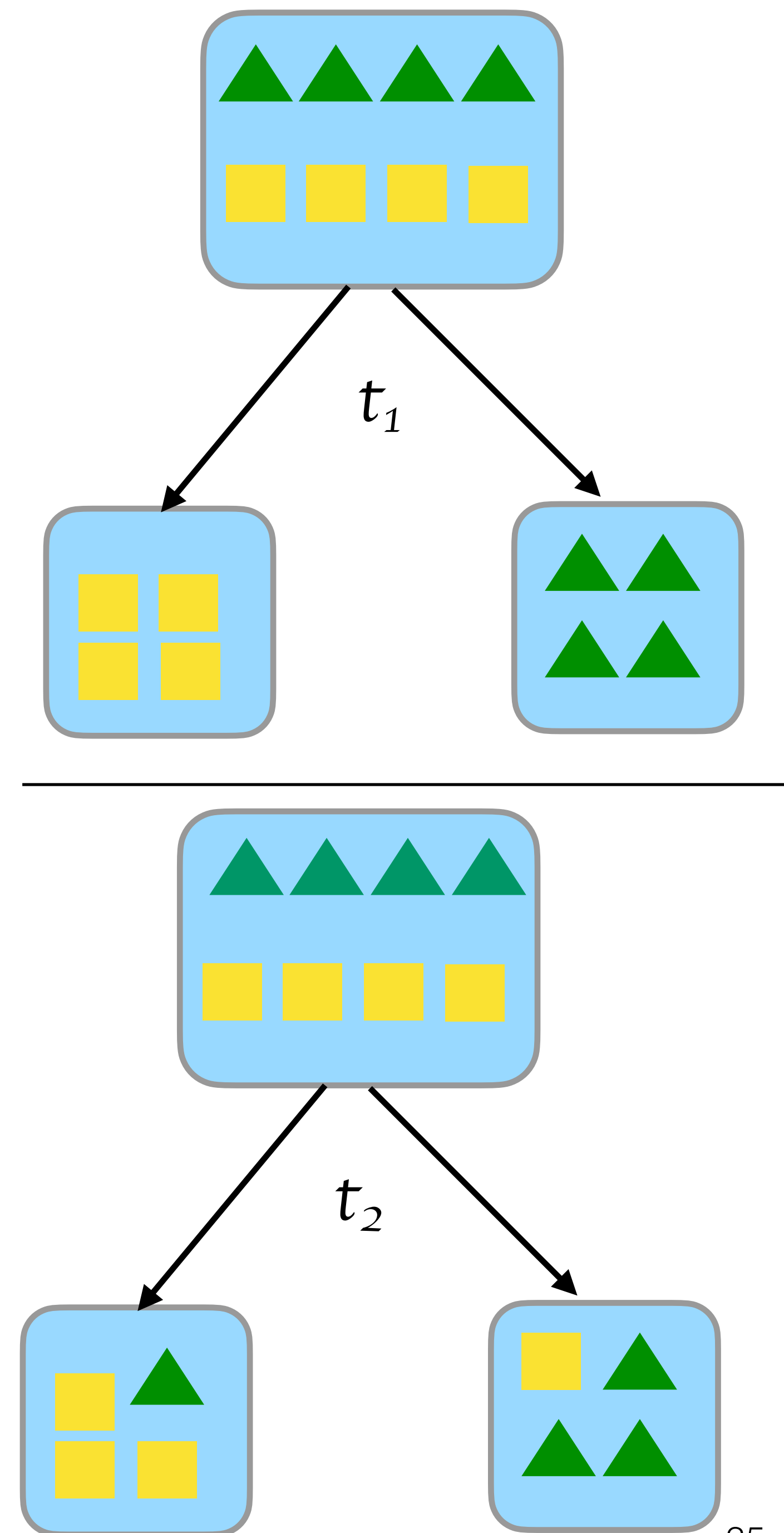
$$\{X \mid X_j \geq t, X \in \mathcal{R}_p\})$$

# Decision Trees

## Split Function

### The goal of split function

- When a tree splits a parent region into two children, we want:
  - Each child to be **purier** (more homogeneous in class labels) than the parent.
  - A metric that **quantifies impurity** and tells us how much “information” the split provides.



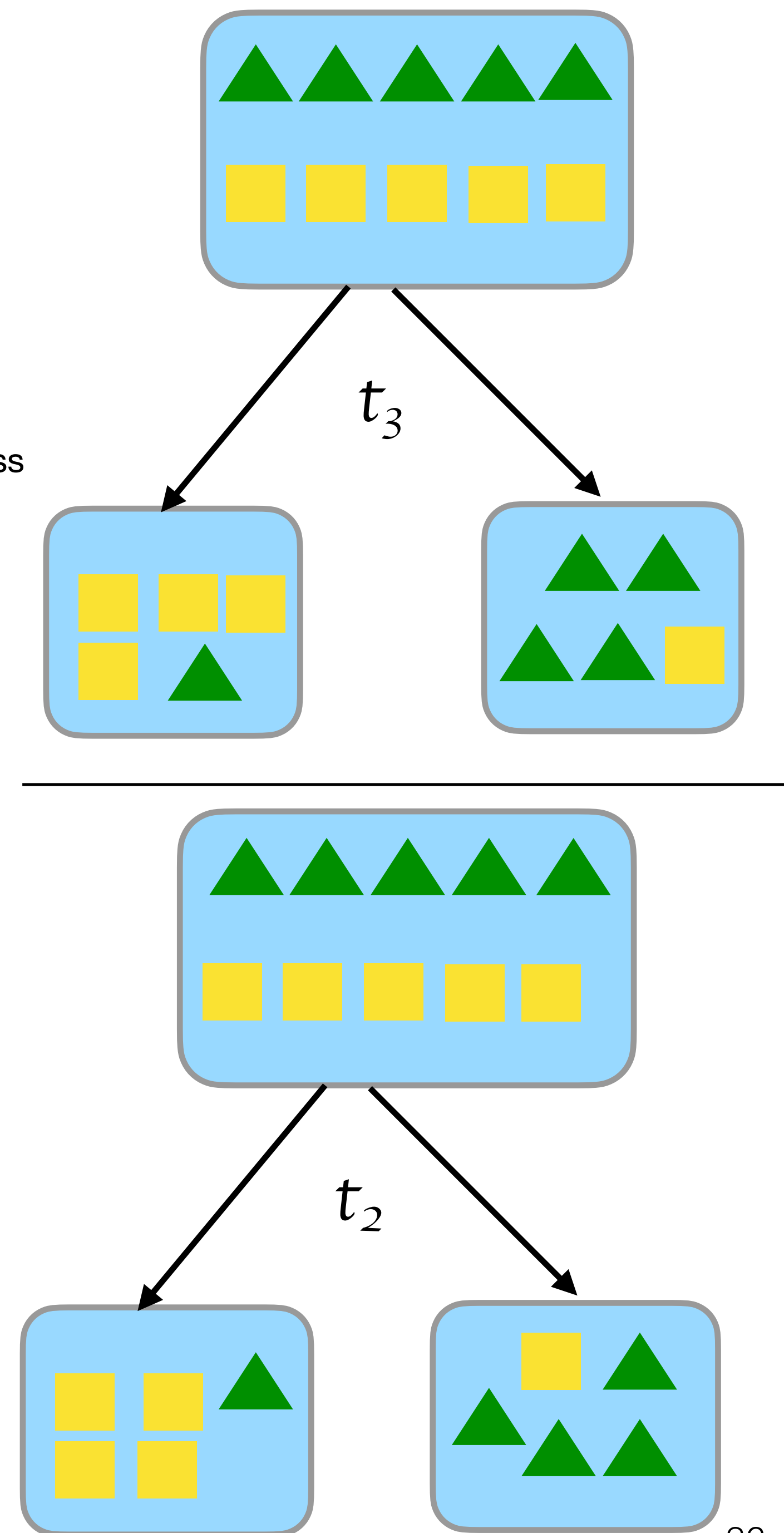
# Decision Trees

## Split Function

$$\text{loss} = 1 - \max_c \underbrace{\hat{p}_c}_{\text{proportion of class } c \text{ in the node}}$$

### Option 1. Misclassification error

- Misclassification loss just counts majority vs. minority class.
- with 90% positives  $\rightarrow$  impurity = 0.1
- Problem: It's **insensitive** to changes in class proportions until the majority class flips.
- It doesn't capture the "quality" of the split well.



# Decision Trees

## Split Function

### Option 2. Cross Entropy

- Entropy is a scientific measure of disorder, randomness, or uncertainty within a system.
- Cross-entropy is a measure of the difference between two probability distributions.
- In machine learning, it serves as a loss function that calculates how wrong a model's predicted probabilities are compared to the true labels, penalizing confident wrong guesses heavily to guide optimization.

# Decision Trees

## Split Function

### Option 2. Cross Entropy

- In machine learning, it serves as a loss function that calculates how wrong a model's predicted probabilities are compared to the true labels, penalizing confident wrong guesses heavily to guide optimization.
- It measures the unpredictability of data.
- A rigged coin that always lands on heads has zero entropy because there is no uncertainty.
- A perfectly fair coin has maximum entropy because the outcome is entirely unpredictable.

# Decision Trees

## Split Function

### Option 2. Cross Entropy

- Cross entropy (a.k.a. Shannon entropy) is defined as:

$$H(p) = - \sum_c \hat{p}_c \log(\hat{p}_c)$$

  
proportion of  
class  $c$  in the node

The minus sign is there simply to make the final result a positive number.

Probabilities are fractions and Logarithms of fractions are negative

- **High entropy:** when classes are evenly mixed (uncertain, impure).
- **Low entropy:** when one class dominates (certain, pure)

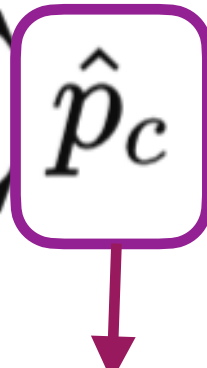
# Decision Trees

## Split Function

### Option 2. Cross Entropy

- Cross entropy (a.k.a. Shannon entropy) is defined as:

$$H(p) = - \sum_c \hat{p}_c \log(\hat{p}_c)$$

  
proportion of  
class  $c$  in the node

- When evaluating a split, we look at:

$$\text{Gain} = H(\text{parent}) - \left( \frac{n_1}{n} H(\text{child}_1) + \frac{n_2}{n} H(\text{child}_2) \right)$$

# Decision Trees

## Split Function

### Option 2. Cross Entropy

- Cross entropy (a.k.a. Shannon entropy)

$$H(p) = - \sum_c \hat{p}_c \log(\hat{p}_c)$$

proportion of class  $c$  in the node

$$\text{Gain} = H(\text{parent}) - \left( \frac{n_1}{n} H(\text{child}_1) + \frac{n_2}{n} H(\text{child}_2) \right)$$

- Why it's good:
  1. **Smooth sensitivity** → Unlike misclassification, entropy smoothly decreases as class purity increases (from 50/50 to 90/10 to 95/5).
  2. **Information-theoretic foundation** → Measures the “amount of uncertainty” in a distribution. Splits reduce uncertainty.
  3. **Rewards highly pure splits** → A split that makes children nearly pure gives a large gain.

# Decision Trees

## Split Function

### Option 2. Cross Entropy

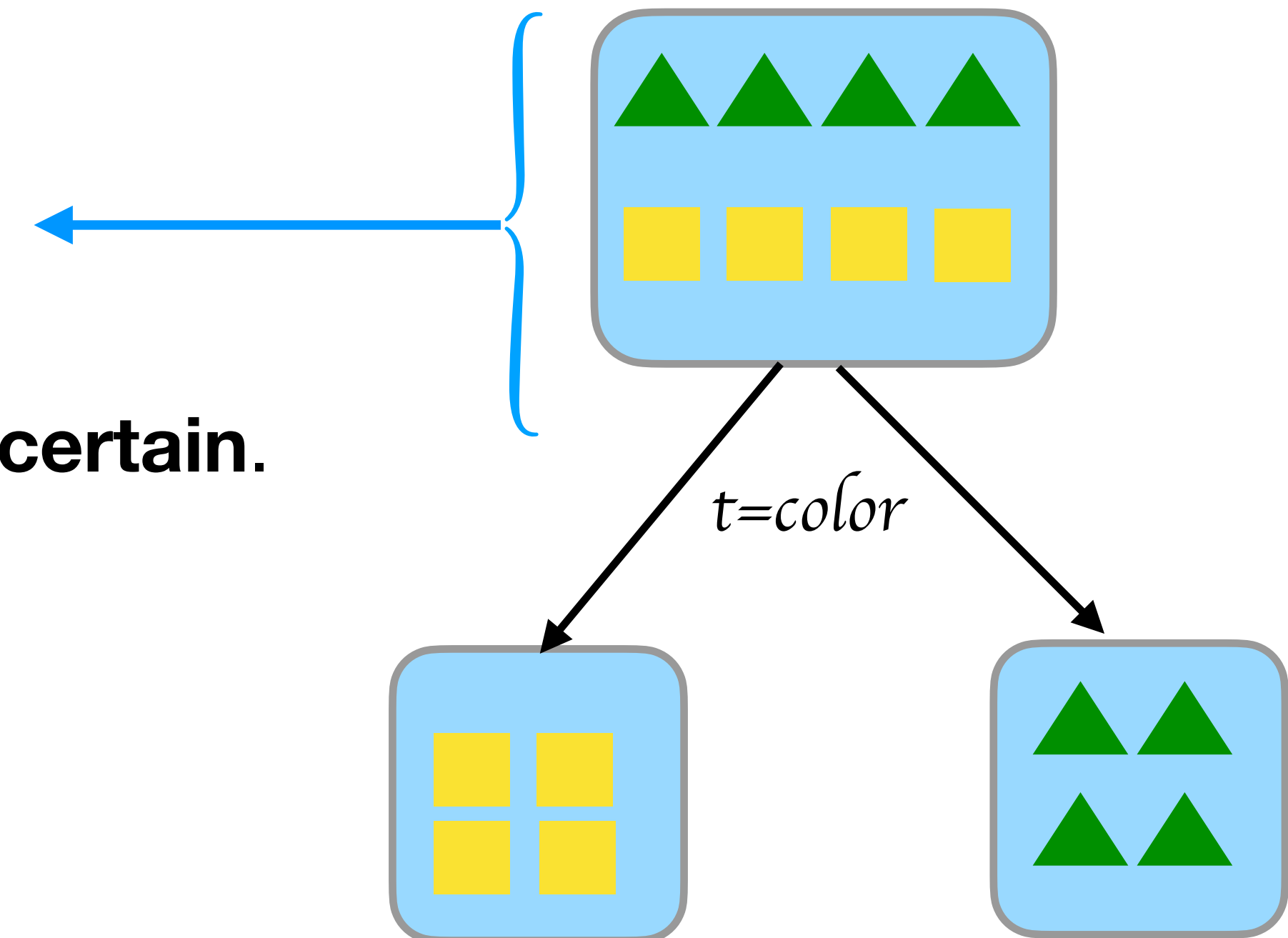
#### Entropy at the Parent Node

$$\mathcal{H} = -(0.5 \log_2 0.5 + 0.5 \log_2 0.5) = 1.0 \text{ (max uncertainty)}$$

- This makes sense: the node is 50/50 → we are **completely uncertain**.

$$H(p) = - \sum_c \underbrace{\hat{p}_c}_{\text{proportion of class } c \text{ in the node}} \log(\hat{p}_c)$$

$$\text{Gain} = H(\text{parent}) - \left( \frac{n_1}{n} H(\text{child}_1) + \frac{n_2}{n} H(\text{child}_2) \right)$$



# Decision Trees

## Split Function

### Option 2. Cross Entropy

$$H(p) = - \sum_c \hat{p}_c \log(\hat{p}_c)$$

proportion of class  $c$  in the node

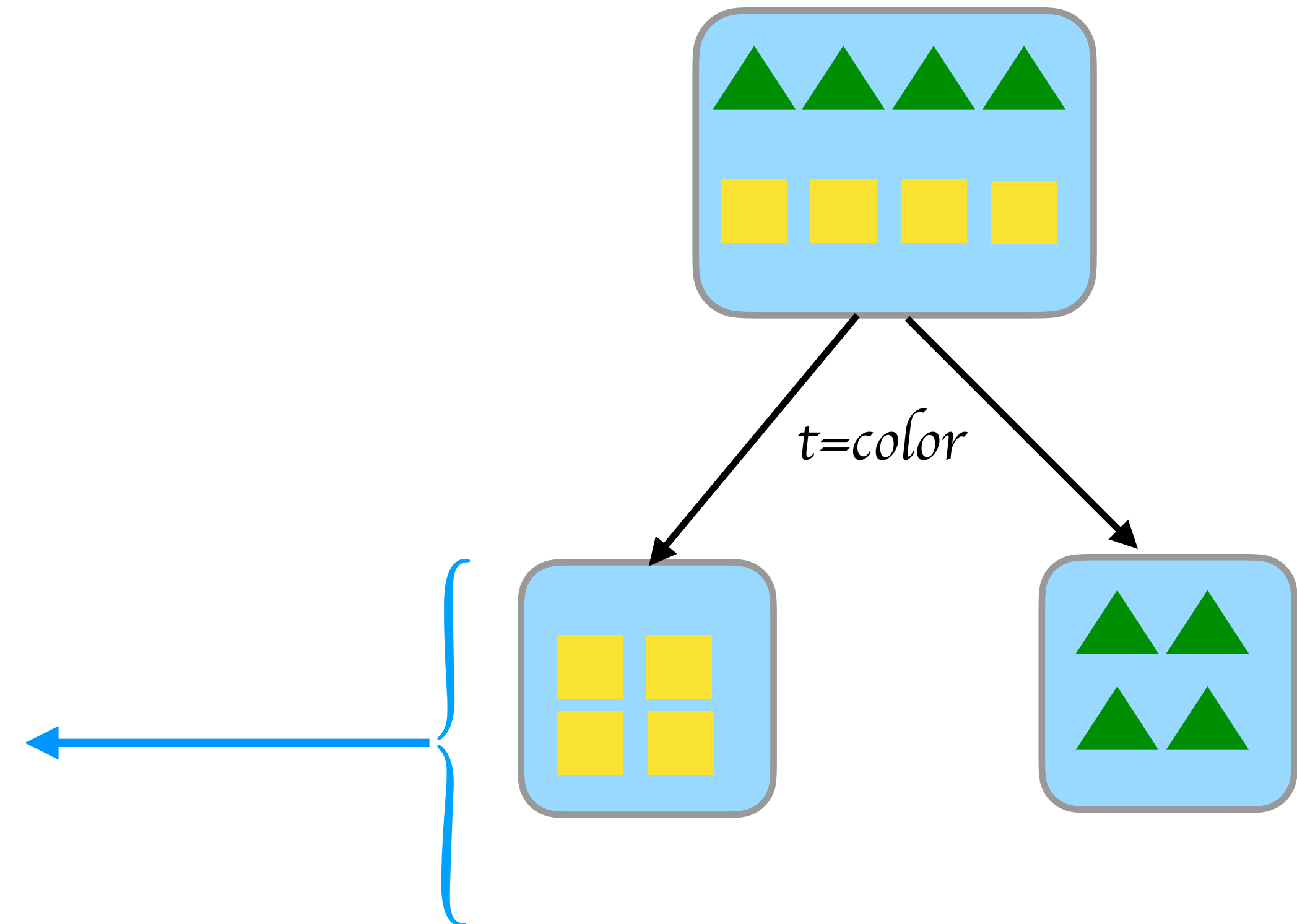
$$\text{Gain} = H(\text{parent}) - \left( \frac{n_1}{n} H(\text{child}_1) + \frac{n_2}{n} H(\text{child}_2) \right)$$

### Entropy at the child Node

$$\mathcal{H} = -(1.0 \log_2 1.0 + 1.0 \log_2 1.0) = 0 \text{ (max uncertainty)}$$

$\log_2 1 = 0$

- Weighted average child entropy = 0.
- Information Gain =  $1.0 - 0 = 1.0 \rightarrow$  perfect split.



# Decision Trees

## Split Function

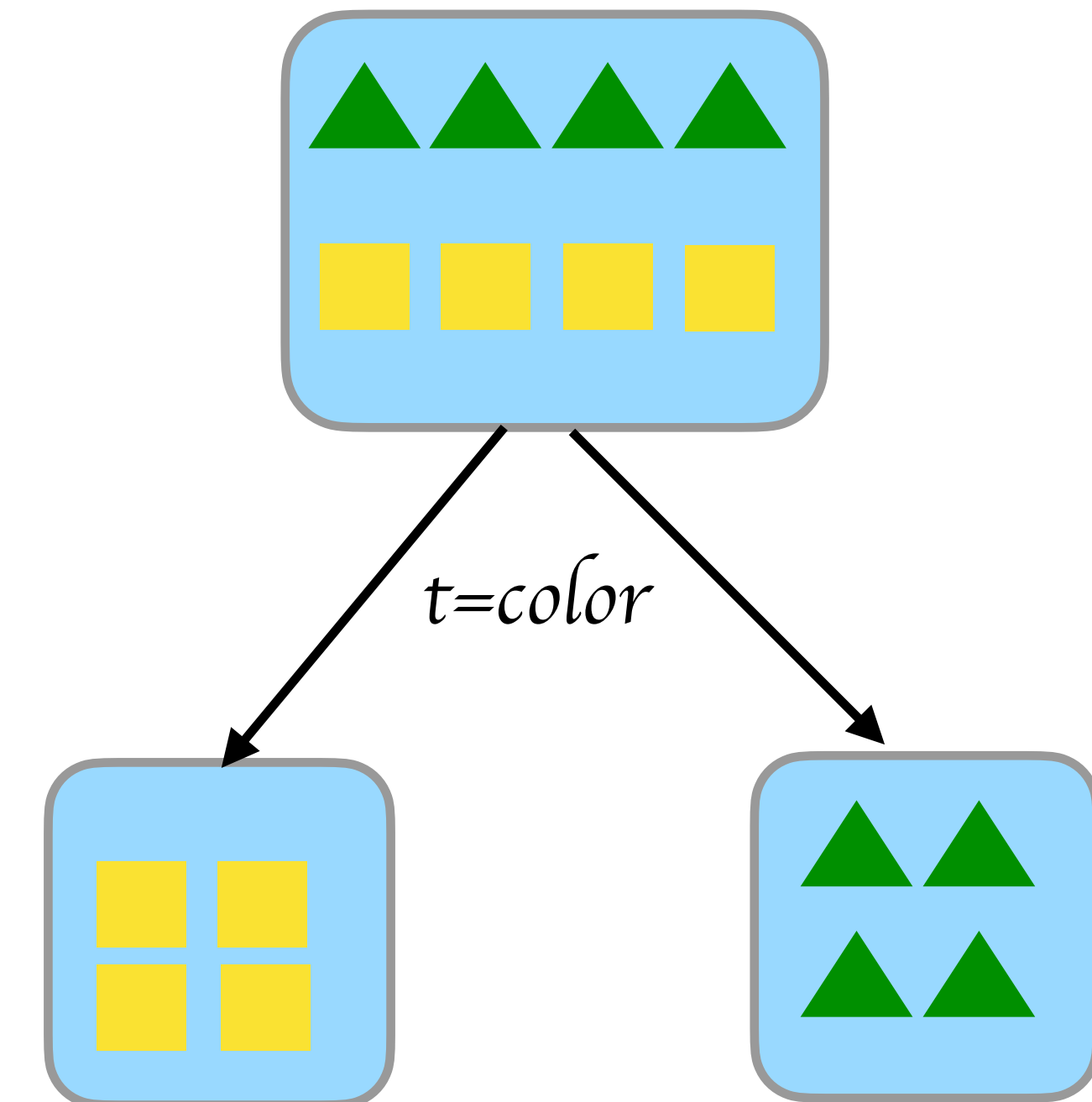
### Option 2. Cross Entropy

- Cross-entropy is **high** when nodes are **mixed**.
- Cross-entropy is **low** when nodes are **pure**.
- A **good split** = a big drop in entropy = we gained “information” about the class.

$$H(p) = - \sum_c \hat{p}_c \log(\hat{p}_c)$$

proportion of class  $c$  in the node

$$\text{Gain} = H(\text{parent}) - \left( \frac{n_1}{n} H(\text{child}_1) + \frac{n_2}{n} H(\text{child}_2) \right)$$



# Decision Trees

## Split Function

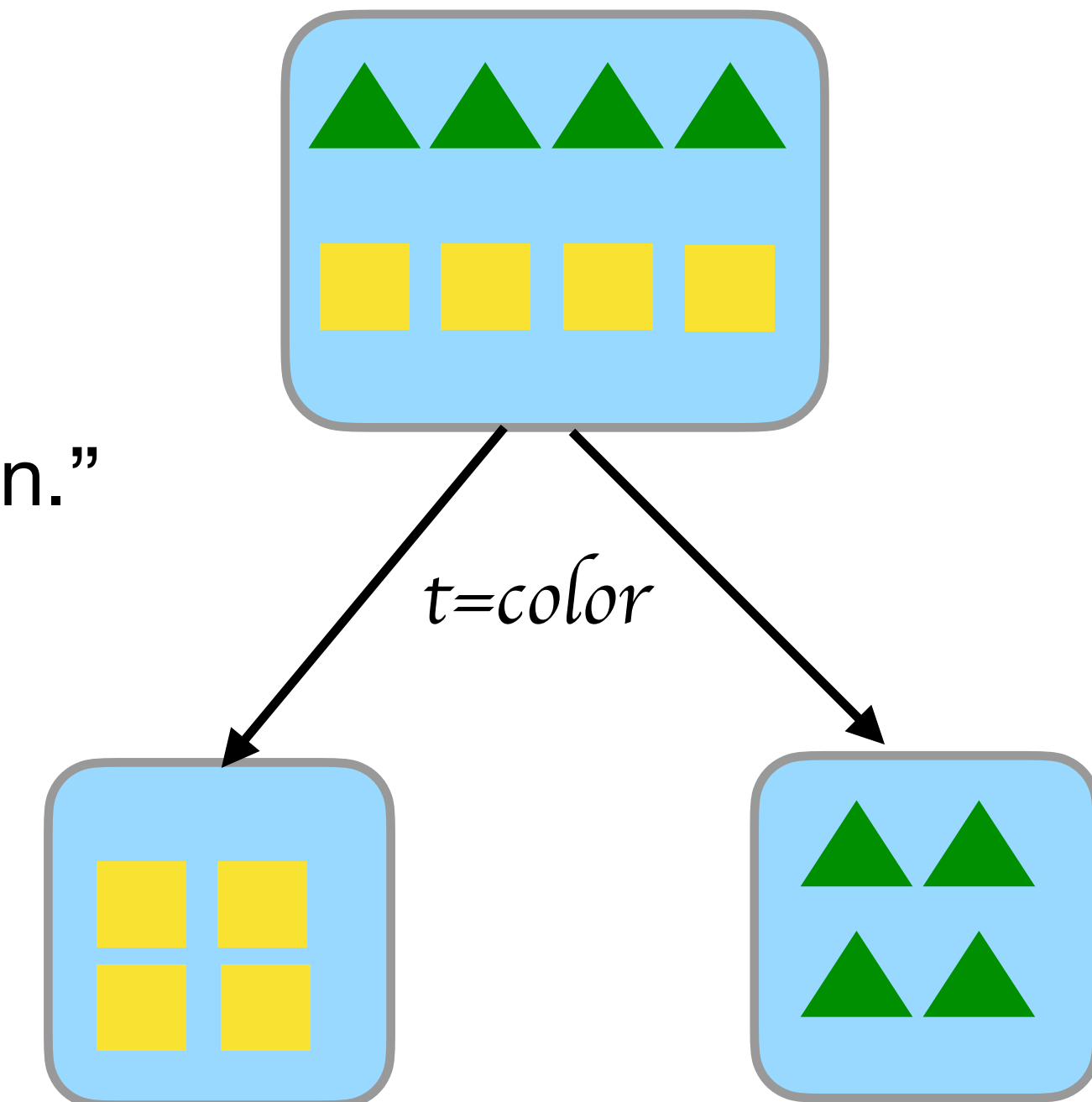
### Option 3. Gini Index

- We just saw:
  - Cross-Entropy measures **uncertainty** in a node.
  - Splits are chosen to **reduce uncertainty** → maximize “information gain.”
- But
  - Cross-Entropy isn't the *only* impurity measure.
  - Another very popular one: **Gini Index**.

$$H(p) = - \sum_c \hat{p}_c \log(\hat{p}_c)$$

proportion of class  $c$  in the node

$$\text{Gain} = H(\text{parent}) - \left( \frac{n_1}{n} H(\text{child}_1) + \frac{n_2}{n} H(\text{child}_2) \right)$$



# Decision Trees

## Split Function

### Option 3. Gini Index

- Gini Index measures the "impurity" or randomness of a group of items.
- Specifically, it calculates the probability that a randomly chosen item from a dataset would be incorrectly labeled if you randomly assigned it a class based on the distribution in that dataset.

# Decision Trees

## Split Function

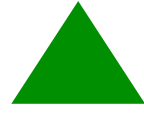
### Option 3. Gini Index

- **Gini = 0.0 (Perfect Purity):** if the node is pure.
- **Gini = 0.5 (Maximum Impurity):** The group is evenly split (50/50) between the two classes.
- **Higher Gini** means more mixed classes.
- Curve shape is similar to entropy, just a bit smoother.
- Gini is slightly faster (no log).

$$G = \sum_c \hat{p}_c (1 - \hat{p}_c) = 1 - \sum_c \hat{p}_c^2$$

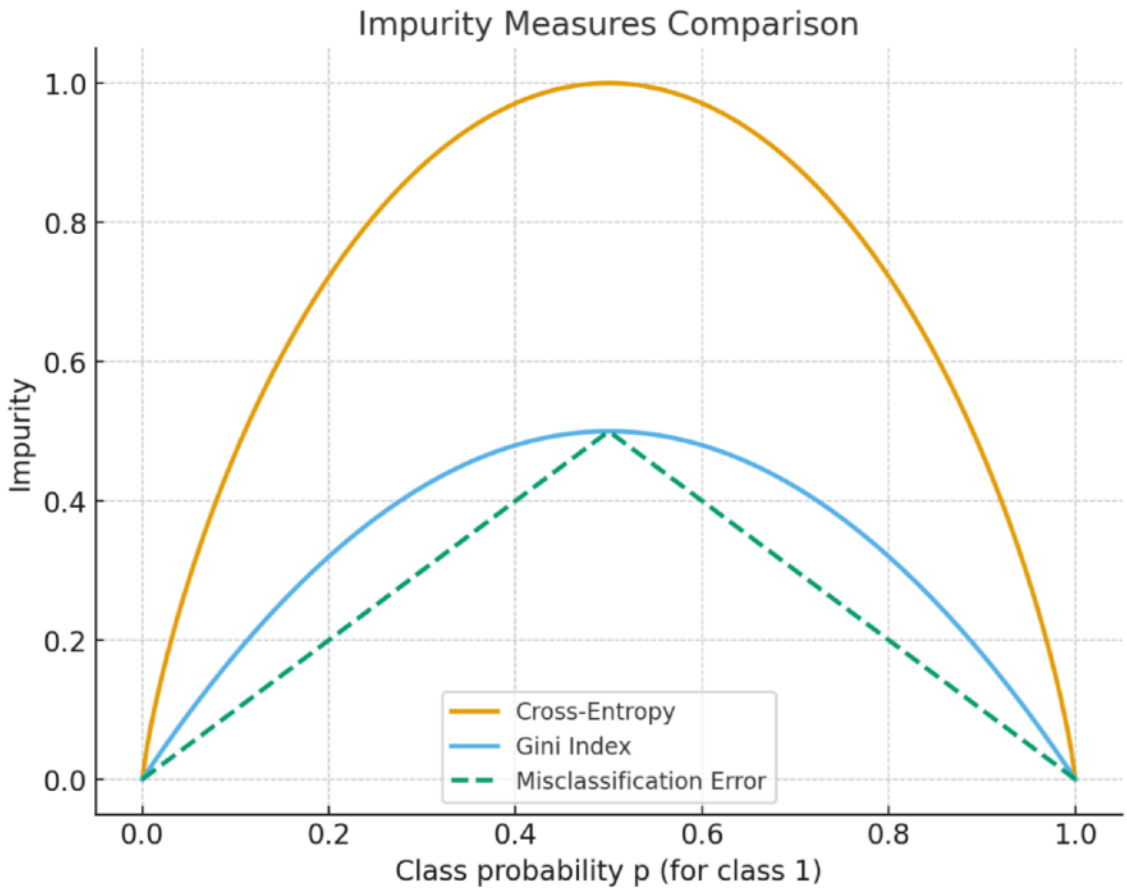
↓  
proportion of class  $c$  in the node

## Intuition

- Think of Gini as the **probability of mislabeling** a sample if you randomly assign labels according to the class distribution in the node.
- Example: If 70% square , 30% triangle  → probability of “wrong guess”  $\approx 0.42$ .

# Decision Trees

## Comparison Table



| Measure                 | Formula                               | Range           | Interpretation                                        | Max at p=0.5  |
|-------------------------|---------------------------------------|-----------------|-------------------------------------------------------|---------------|
| Misclassification Error | $1 - \max(p, 1-p)$                    | $[0, 0.5]$      | Probability of incorrect prediction                   | 0.5           |
| Gini Index              | $\sum p_c (1 - p_c) = 1 - \sum p_c^2$ | $[0, 0.5]$      | Probability of misclassification if assigned randomly | 0.5           |
| Cross-Entropy (Entropy) | $-\sum p_c \log_2(p_c)$               | $[0, \log_2 K]$ | Expected “uncertainty” in bits (info theory)          | 1.0 (for K=2) |

# Decision Trees

## Comparison impurity measures

- All are 0 when the node is pure (one class only).
- All are maximized when classes are evenly split.
-  • Misclassification is coarse (not sensitive to changes).
-  • Gini and Entropy are smoother, more sensitive, and often give similar splits.

# Example

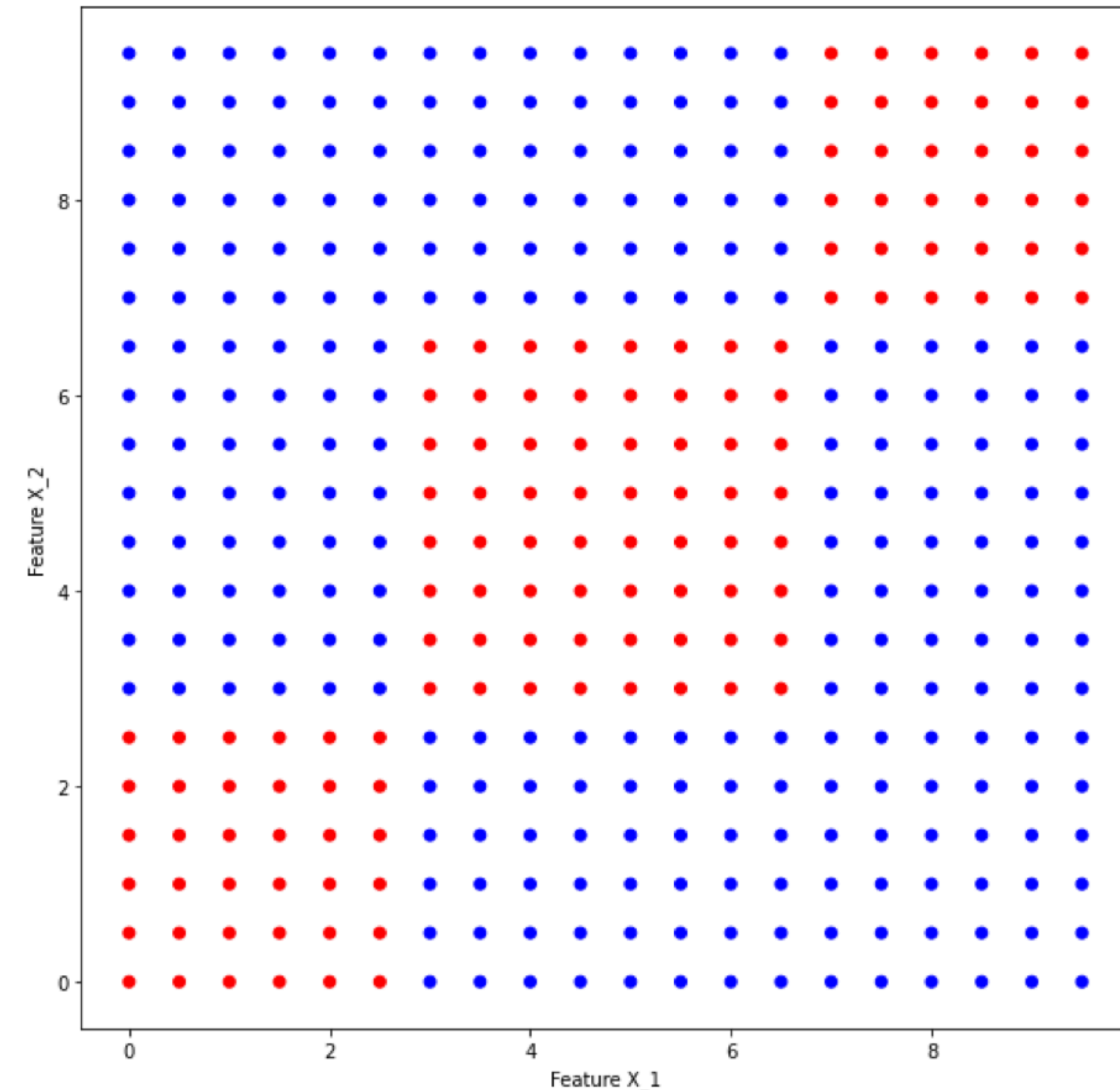
Consider the dataset below with 400 total points consisting of three clusters of red points with 36, 64, and 36 points going from bottom left to top right.

The mutual information represents how much we gain, in terms of reducing the entropy, from knowing something about an attribute:

$$I(Y;X) = H(Y) - H(Y|X),$$

where  $H(Y|X) = \sum_k p(X = k)H(Y | X = k)$  represents the conditional entropy.

1. Starting with the base predictor, what is the information gain (or mutual information) of the split  $X_1 < 3$ ?



$$H(p) = - \sum_c \hat{p}_c \log(\hat{p}_c)$$

# Example

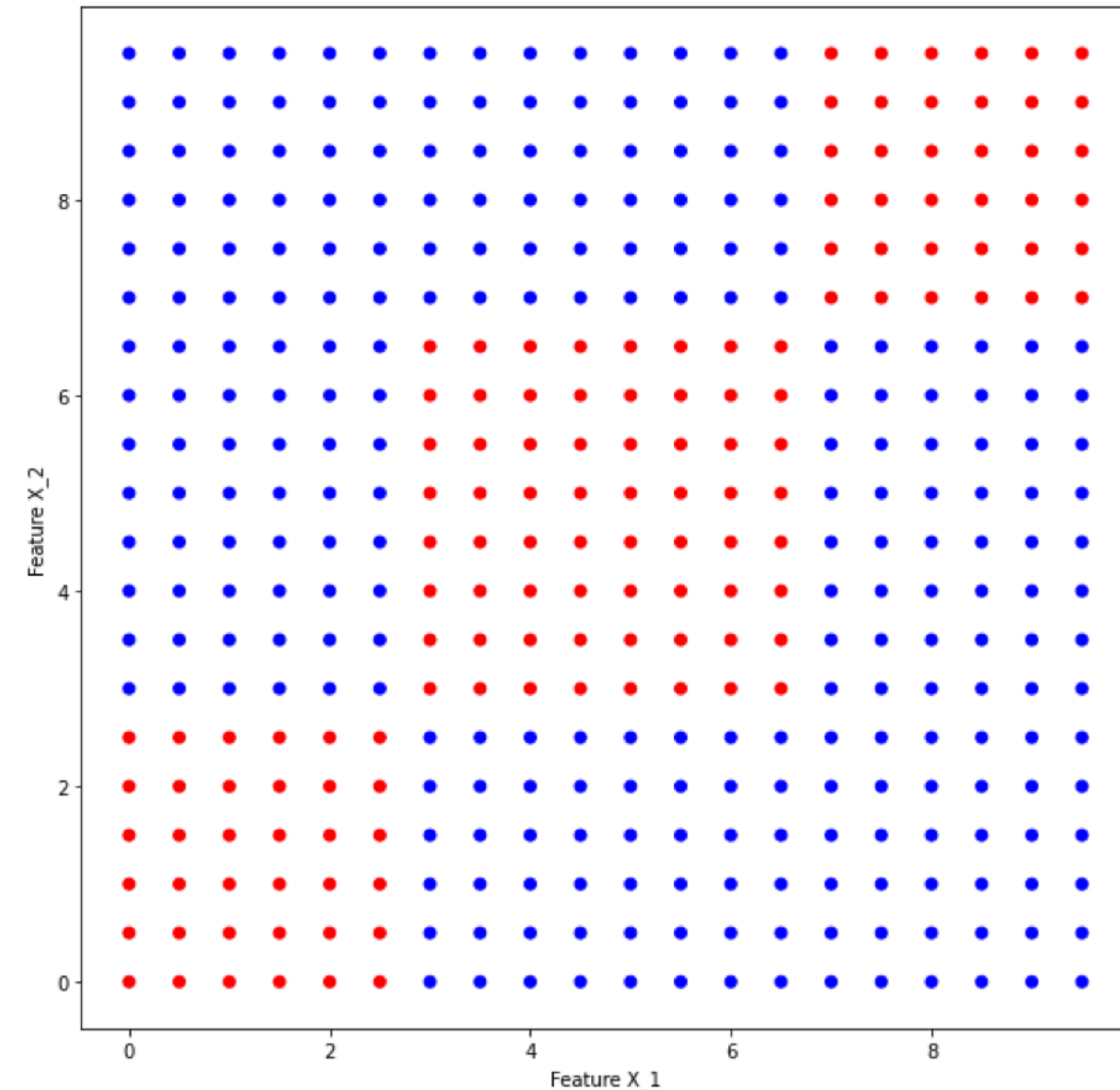
Consider the dataset below with 400 total points consisting of three clusters of red points with 36, 64, and 36 points going from bottom left to top right.

The mutual information represents how much we gain, in terms of reducing the entropy, from knowing something about an attribute:

$$I(Y;X) = H(Y) - H(Y|X),$$

where  $H(Y|X) = \sum_k p(X = k)H(Y | X = k)$  represents the conditional entropy.

2. What is the mutual information of the split  $X_2 < 3$  (we are performing this split AFTER the previous split, i.e. on the points with  $X_1 < 3$ ).



$$H(p) = - \sum_c \hat{p}_c \log(\hat{p}_c)$$

# Example

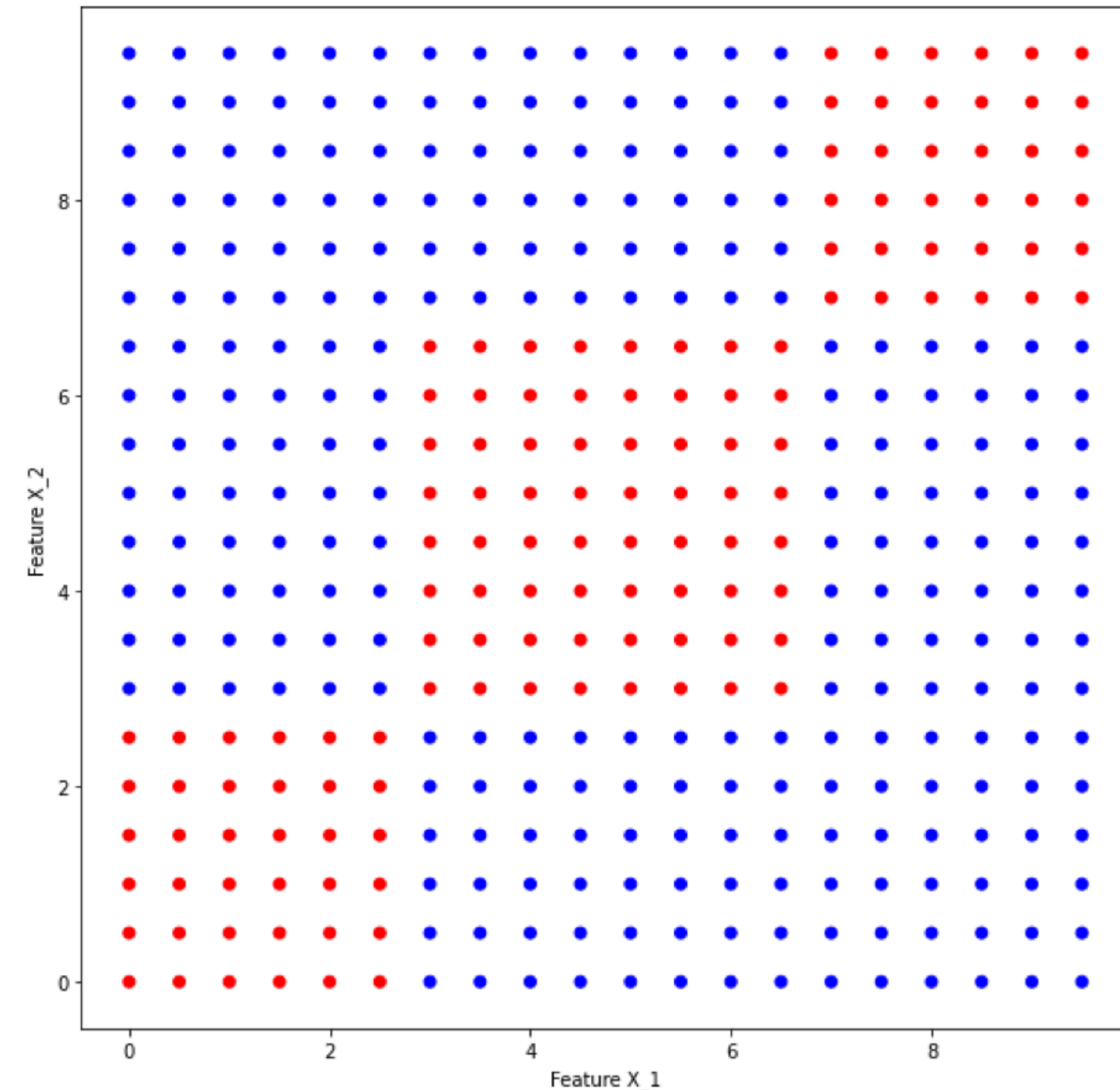
Consider the dataset below with 400 total points consisting of three clusters of red points with 36, 64, and 36 points going from bottom left to top right.

The mutual information represents how much we gain, in terms of reducing the entropy, from knowing something about an attribute:

$$I(Y;X) = H(Y) - H(Y|X),$$

where  $H(Y|X) = \sum_k p(X = k)H(Y | X = k)$  represents the conditional entropy.

3. Draw out a decision tree that could perfectly classify the points in the figure from Question 2. Why can we exactly learn the decision boundary in this case?



$$H(p) = - \sum_c \hat{p}_c \log(\hat{p}_c)$$



**15 minutes break**

# Regression Trees

# Regression Trees

## Motivation

- Decision trees — Classification trees: predict a class label.
- What if the target  $y$  is **continuous**?
  - Example: house prices, customer spending, etc.
- We extend the same tree idea:
  - **Binning (Discretization)**
  - **Regression Trees**

# Decision Trees

## Binning (Discretization)

Binning involves grouping continuous data into a smaller number of "bins" or intervals. This process transforms continuous variables into categorical or ordinal ones.

- **Example:**  
Imagine a dataset of customer ages (a continuous variable).
  - **Equal-width binning:** Ages could be grouped into bins "0-18," "19-30," "31-50," and "51+." Each bin has the same range of values.
  - **Equal-frequency binning:** Alternatively, ages could be grouped so that each bin contains roughly the same number of customers, even if the age ranges differ.
- **Purpose:**  
Binning can simplify data, reduce noise, and make it easier to analyze or use in models that prefer categorical inputs. It can also be a pre-processing step for certain algorithms.

# Decision Trees

## Example Binning: Predicting Fruit Sweetness

Say we have a continuous feature: **sugar\_content** (grams of sugar per 100g).  
Values for 10 fruits:

[3.1, 3.5, 4.0, 4.2, 6.8, 7.1, 7.3, 9.0, 9.2, 9.5]

### Without Binning

- The tree algorithm could test **every possible threshold** between consecutive values:
  - 3.3, 3.75, 4.1, 6.95, 7.2, 8.15, 9.1, etc.
- That's **9 possible splits** for just this one feature.
- With many features and thousands of samples, this becomes costly.

# Decision Trees

## Example Binning: Predicting Fruit Sweetness

Say we have a continuous feature: **sugar\_content** (grams of sugar per 100g).

Values for 10 fruits: [3.1, 3.5, 4.0, 4.2, 6.8, 7.1, 7.3, 9.0, 9.2, 9.5]

### With Binning

- The We define a smaller set of **bins** (intervals): Example: 3 bins

1. Low sugar:  $\leq 5$
2. Medium sugar:  $5 < x \leq 8$
3. High sugar:  $> 8$

So our values become:

Low = [3.1, 3.5, 4.0, 4.2]

Medium = [6.8, 7.1, 7.3]

High = [9.0, 9.2, 9.5]

Now the decision tree only considers **2 split points**:

- Low vs Medium/High
- Low/Medium vs High

# Decision Trees

## Binning: Split points

Ways to choose split points for bins:

1. Equal-width (divide range into equal intervals).
2. Equal-frequency (quantiles).
3. Supervised (data-driven: thresholds that improve impurity/MSE).
4. Domain knowledge (expert-based thresholds).

# Decision Trees

## Binning: Split points

Ways to choose split points for bins:

1. **Equal-width (divide range into equal intervals)**
  - Divide the feature range into intervals of equal size.
  - Example: Sugar content values from 0–10 → bins of width 2 → [0–2], [2–4], [4–6], ...
  - **Simple** but doesn't consider data distribution.

# Decision Trees

## Binning: Split points

Ways to choose split points for bins:

1. Equal-width (divide range into equal intervals).
2. **Equal-frequency (quantiles)**
  - Each bin has (roughly) the same number of samples.
  - Example: If we have 100 samples and choose 4 bins → each bin has ~25 values.
  - **Advantage:** avoids bins with very few samples.
  - Often based on quartiles, deciles, etc.

# Decision Trees

## Binning: Split points

Ways to choose split points for bins:

1. Equal-width (divide range into equal intervals).
2. Equal-frequency (quantiles).
3. **Supervised (data-driven: thresholds that improve impurity/MSE)**
  - Choose bin boundaries that **maximize split quality** (reduce impurity/MSE the most).
  - E.g., in CART, the tree itself evaluates every possible threshold, but you can **pre-bin** into candidate thresholds.
  - Common trick:
    - Sort values of a continuous feature.
    - Compute potential splits only at **midpoints between unique values** where the target label changes.
    - This is what scikit-learn's decision tree does under the hood.

# Decision Trees

## Binning: Split points

Ways to choose split points for bins:

### **4. Domain knowledge (expert-based thresholds).**

Use meaningful thresholds from the problem context.

Example:

- Sugar content: Low ( $<5\text{g}$ ), Medium ( $5\text{--}10\text{g}$ ), High ( $>10\text{g}$ ).
- Age: Child ( $<13$ ), Teen ( $13\text{--}19$ ), Adult ( $20\text{--}65$ ), Senior ( $65+$ ).

# Regression Trees

- Decision trees — Regression trees: predict a continuous value.
  - Binning (Discretization)
  - **Regression Trees**

# Regression Trees

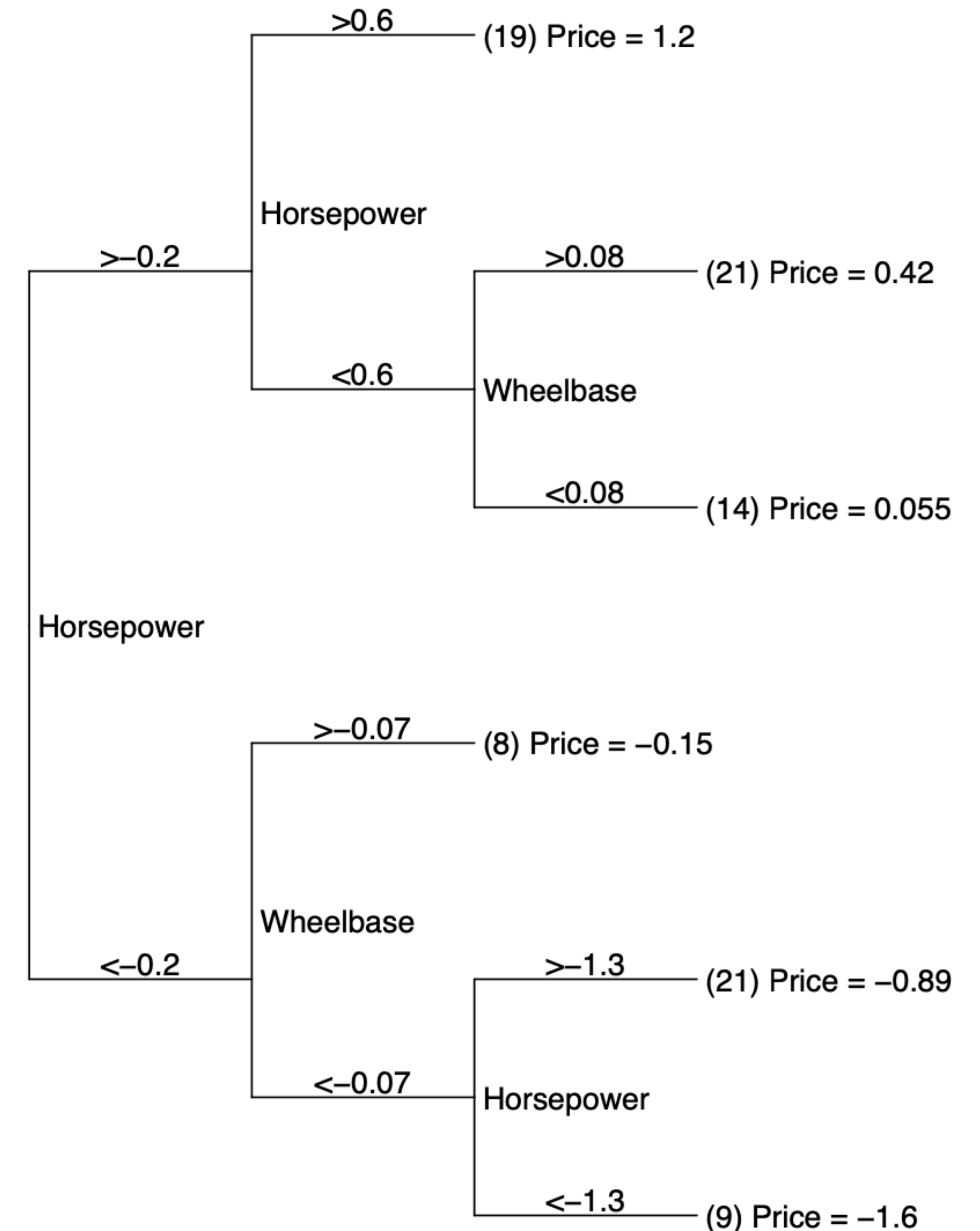
## Motivation

- Idea: Partition the feature space into regions  $\mathcal{R}_1, \mathcal{R}_2, \mathcal{R}_3, \dots, \mathcal{R}_m$

- For a new input  $x$ , predict:

$$\hat{y}(x) = \text{average of training values in the region containing } x$$

- Leaves hold a **numeric prediction** instead of a majority class



# Regression Trees

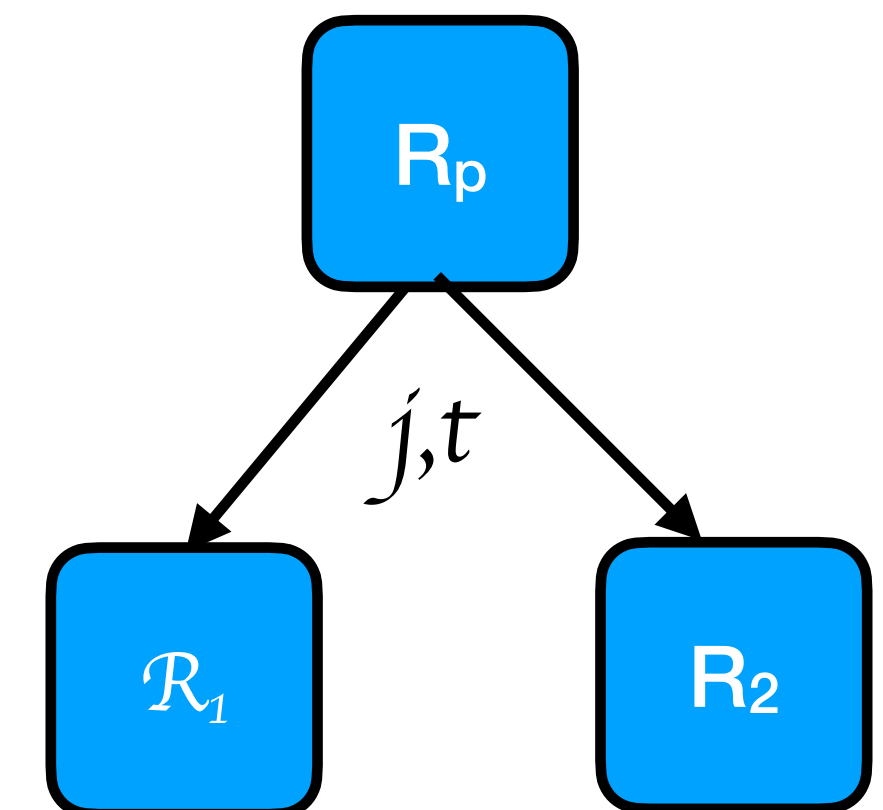
## Objective & Split Function

- Idea: For any region  $R$  (node), let
  - $\hat{y}_R = \frac{1}{|R|} \sum_{i \in R} y_i$  (leaf prediction = mean),
  - $\text{SSE}(R) = \sum_{i \in R} (y_i - \hat{y}_R)^2$ . SSE: Sum of Squared Errors
- At a parent node  $R_p$ , for a candidate split on

- feature  $j$  at threshold  $t$  that

$$\Delta(j, t) = \text{SSE}(R_p) - (\text{SSE}(R_1) + \text{SSE}(R_2))$$

- creates children  $R_1, R_2$
- A **high SSE** means the numbers in the group are vastly different from each other.
- A **low SSE** means the numbers are tightly clustered around the average.



# Regression Trees

## Objective & Split Function

- Idea: For any region  $R$  (node), let

- $\hat{y}_R = \frac{1}{|R|} \sum_{i \in R} y_i$  (leaf prediction = mean),
- $\text{SSE}(R) = \sum_{i \in R} (y_i - \hat{y}_R)^2$ .      SSE: Sum of Squared Errors

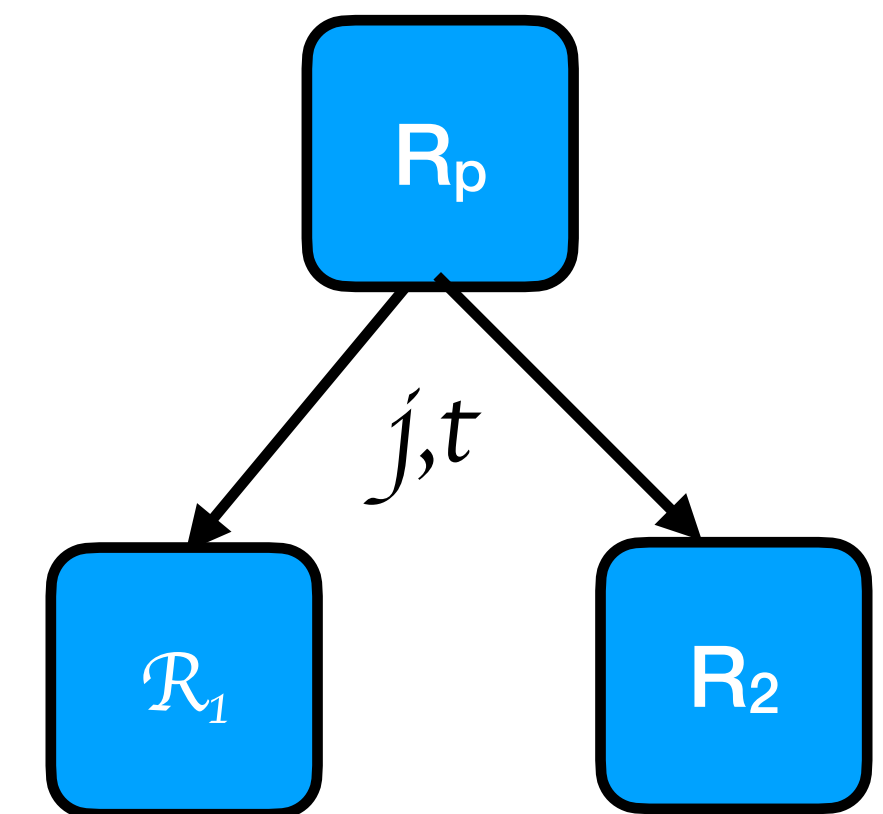
- At a parent node  $R_p$ , for a candidate split on

- feature  $j$  at threshold  $t$  that  $\Delta(j, t) = \text{SSE}(R_p) - (\text{SSE}(R_1) + \text{SSE}(R_2))$

- creates children  $R_1, R_2$

- Equivalently, **minimizes**  $\text{SSE}(R_1) + \text{SSE}(R_2)$ .

- This is exactly the “minimize total within-leaf sum of squares” criterion.



# Regression Trees

## CART Algorithm

- Classification and Regression Trees (CART) is a decision tree algorithm that is used for both classification and regression tasks.
- Regression CART works by splitting the training data recursively into smaller subsets based on specific criteria.
- The objective is to split the data in a way that minimizes the residual reduction in each subset.

# Regression Trees

## CART Algorithm

Pseudo-code of the CART algorithm

```
d = 0, endtree = 0
Node(0) = 1, Node(1) = 0, Node(2) = 0
while endtree < 1
    if Node(2d-1) + Node(2d) + .... + Node(2d+1-2) = 2 - 2d+1
        endtree = 1
    else
        do i = 2d-1, 2d, .... , 2d+1-2
            if Node(i) > -1
                Split tree
            else
                Node(2i+1) = -1
                Node(2i+2) = -1
            end if
        end do
    end if
    d = d + 1
end while
```

# Decision Trees

## CART algorithm

### 1. Residual Reduction

- When a regression tree makes a prediction in a region (leaf), it predicts the **average value** of all points in that region.
- The “residual” is just **how far off each point is from that average**.
- The **residual error** = squared difference between actual and predicted values.
- Before the split: compute how “spread out” the values are around their average.
- After the split: compute how “spread out” the values are in the two smaller groups.
- The difference = how much error we reduced by splitting.

#### Residual reduction Example:

- Imagine sorting candy by size. If you dump all candies in one jar, the “average size” is not very useful.
- If you split them into “small” and “large” jars, the average in each jar is much closer to the candies inside — that’s **residual reduction**.

# Decision Trees

## CART algorithm

### 2. Splitting Criteria

- At each node, CART asks: “Which feature and threshold should I split on?”
- To decide, it tries **every possible split** (*Greedy algorithm*) and calculates how much it reduces residual error.
- The split that gives the **biggest reduction in error** is chosen.
- This is repeated recursively: keep splitting nodes into smaller and smaller regions until we hit a **stopping rule** (e.g., max depth, or too few points).
- **Splitting criteria** = pick the split that makes predictions more accurate by reducing the leftover error as much as possible.

#### Splitting Criteria Example:

- Imagine sorting candy by size. If you dump all candies in one jar, the “average size” is not very useful.
- You choose the split (e.g., size = 3cm) that makes the candies in each jar as similar as possible — that’s the **splitting criterion**.

# CART Algorithm

## Example Predicting Apple Weights

We have 6 apples with weights (grams):  $y=[100,102,98,200,198,202]$

**Step 1.** Before splitting (all apples together)

Mean weight =

$$\hat{y} = \frac{100 + 102 + 98 + 200 + 198 + 202}{6} = 150$$

Residual error = sum of squared differences:

$$\begin{aligned} \text{SSE}(\text{parent}) &= (100 - 150)^2 + (102 - 150)^2 + (98 - 150)^2 + (200 - 150)^2 + (198 - 150)^2 + (202 - 150)^2 \\ &= 2500 + 2304 + 2704 + 2500 + 2304 + 2704 = 15,016 \end{aligned}$$

# CART Algorithm

## Example Predicting Apple Weights

We have 6 apples with weights (grams):  $y=[100, 102, 98, 200, 198, 202]$

**Step 1.** Before splitting (all apples together) Mean weight =

$$\hat{y} = \frac{100 + 102 + 98 + 200 + 198 + 202}{6} = 150$$

**Step 2.** After splitting (say split at 150g)

Left group =  $[100, 102, 98] \rightarrow \text{mean} = 100$

$$\text{SSE}(\text{left}) = (100-100)^2 + (102-100)^2 + (98-100)^2 = 0 + 4 + 4 = 8$$

Right group =  $[200, 198, 202] \rightarrow \text{mean} = 200$

$$\text{SSE}(\text{right}) = (200-200)^2 + (198-200)^2 + (202-200)^2 = 0 + 4 + 4 = 8$$

$$\text{SSE}(\text{children})=8+8=16$$

# CART Algorithm

## Example Predicting Apple Weights

We have 6 apples with weights (grams):  $y=[100, 102, 98, 200, 198, 202]$

**Step 1.** Before splitting (all apples together)

**Step 2.** After splitting (say split at 150g)

**Step 3.** Residual Reduction

$$\Delta = \text{SSE}(\text{parent}) - \text{SSE}(\text{children}) = 15,016 - 16 = 15,000$$

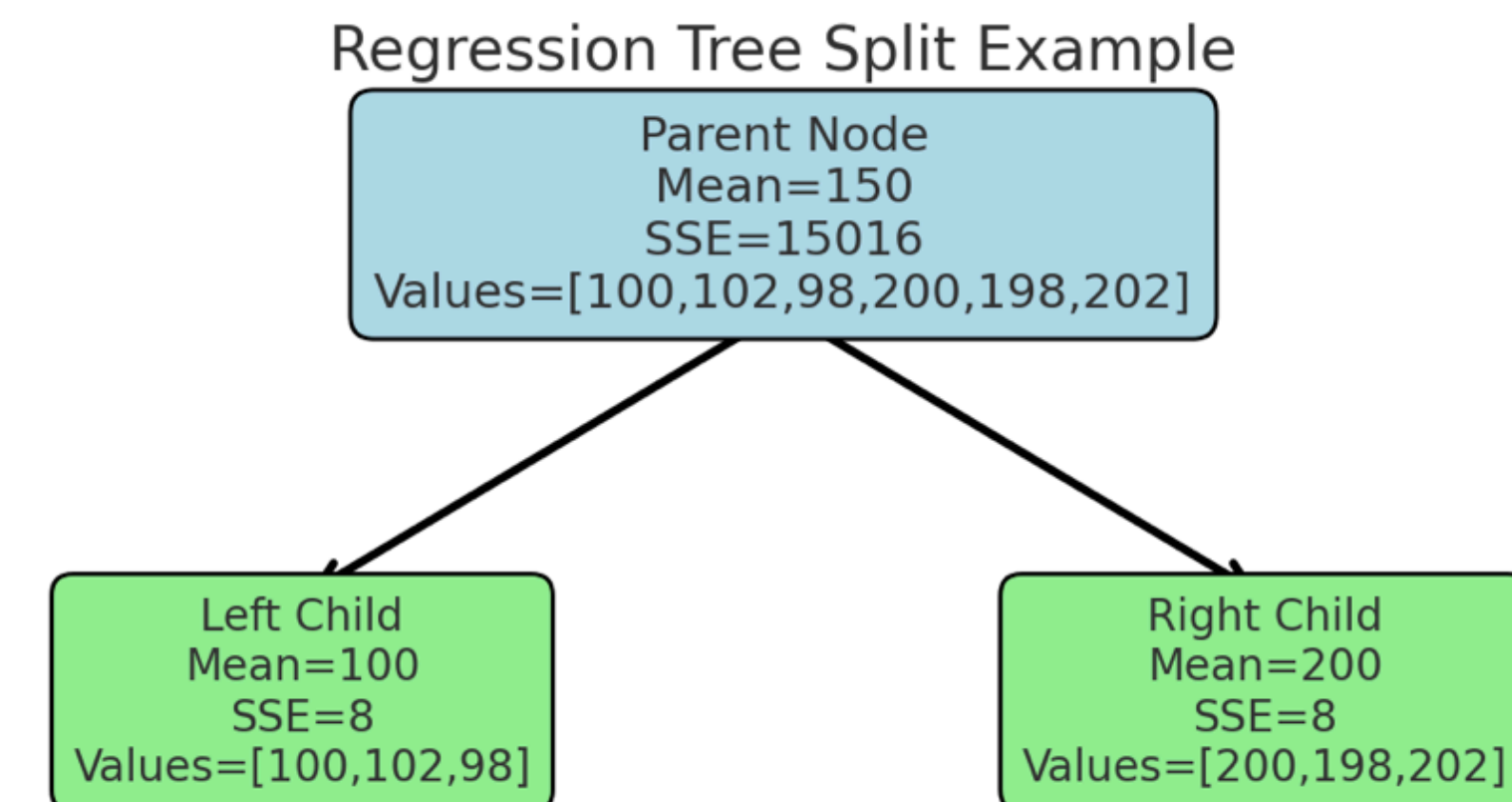
**SSE at the Parent Node:** We had all 6 apple weights in one group

$[100, 102, 98, 200, 198, 202]$ . Mean = 150.

$$\text{SSE}(\text{parent}) = (100-150)^2 + (102-150)^2 + (98-150)^2 + (200-150)^2 + (198-150)^2 + (202-150)^2 = 15,016$$

This is big, because predicting 150 is a terrible fit for numbers clustered near 100 and 200.

**SSE at the Children — After splitting at 150g:**



# CART Algorithm

## Example Predicting Apple Weights

We have 6 apples with weights (grams):  $y=[100, 102, 98, 200, 198, 202]$

**Step 1.** Before splitting (all apples together)

**Step 2.** After splitting (say split at 150g)

**Step 3.** Residual Reduction

**SSE at the Parent Node:**

**SSE at the Children — After splitting at 150g:**

- Left child:  $[100, 102, 98]$ , mean = 100

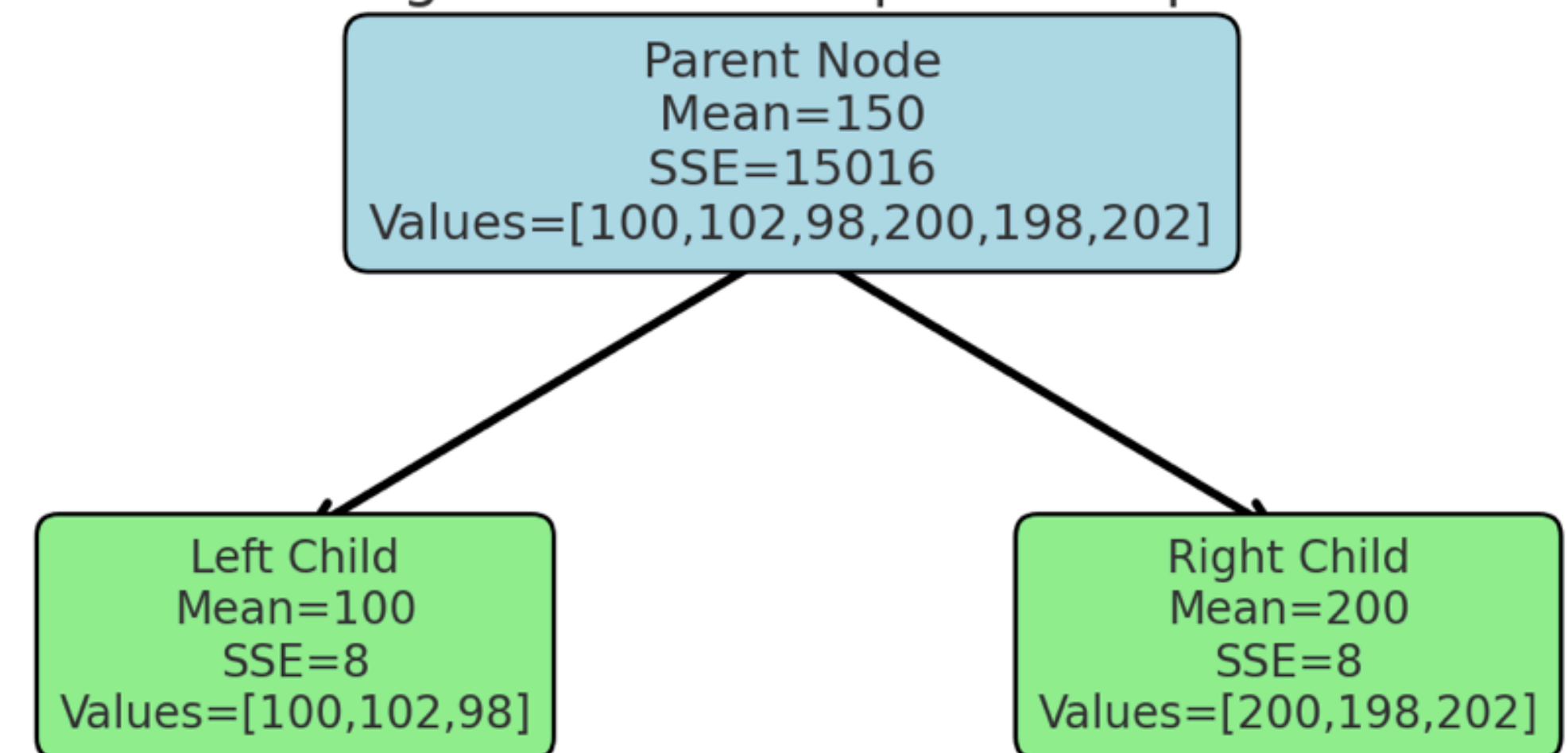
$$\text{SSE}(\text{left}) = (100 - 100)^2 + (102 - 100)^2 + (98 - 100)^2 = 8$$

- Right child:  $[200, 198, 202]$ , mean = 200

$$\text{SSE}(\text{right}) = (200 - 200)^2 + (198 - 200)^2 + (202 - 200)^2 = 8$$

$$\text{SSE}(\text{children}) = 8 + 8 = 16$$

Regression Tree Split Example



# CART Algorithm

## Example Predicting Apple Weights

We have 6 apples with weights (grams):  $y=[100, 102, 98, 200, 198, 202]$

**Step 1.** Before splitting (all apples together)

**Step 2.** After splitting (say split at 150g)

**Step 3.** Residual Reduction

$$\Delta = \text{SSE}(\text{parent}) - \text{SSE}(\text{children}) = 15,016 - 16 = 15,000$$

**SSE at the Parent Node: 15016**

**SSE at the Children — After splitting at 150g:**

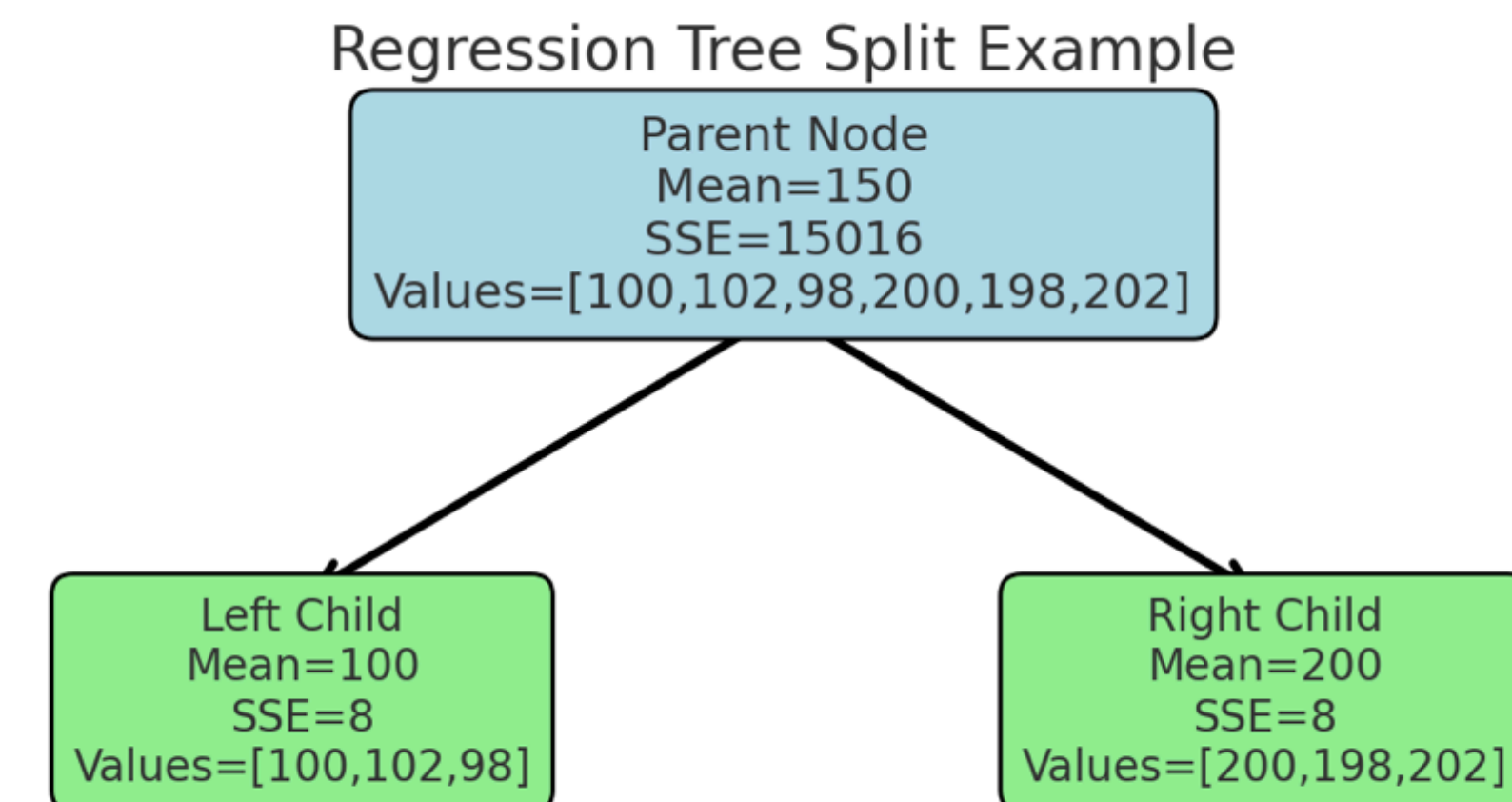
- Left child:  $[100, 102, 98]$ , mean = 100

$$\text{SSE}(\text{left}) = (100 - 100)^2 + (102 - 100)^2 + (98 - 100)^2 = 8$$

- Right child:  $[200, 198, 202]$ , mean = 200

$$\text{SSE}(\text{right}) = (200 - 200)^2 + (198 - 200)^2 + (202 - 200)^2 = 8$$

$$\text{SSE}(\text{children}) = 8 + 8 = 16$$



# Ensemble Models

# Transition to Ensembles

## Motivation

Single decision trees:

- Easy to interpret.
- Flexible (handle nonlinearities).
- BUT: prone to **overfitting** (high variance).

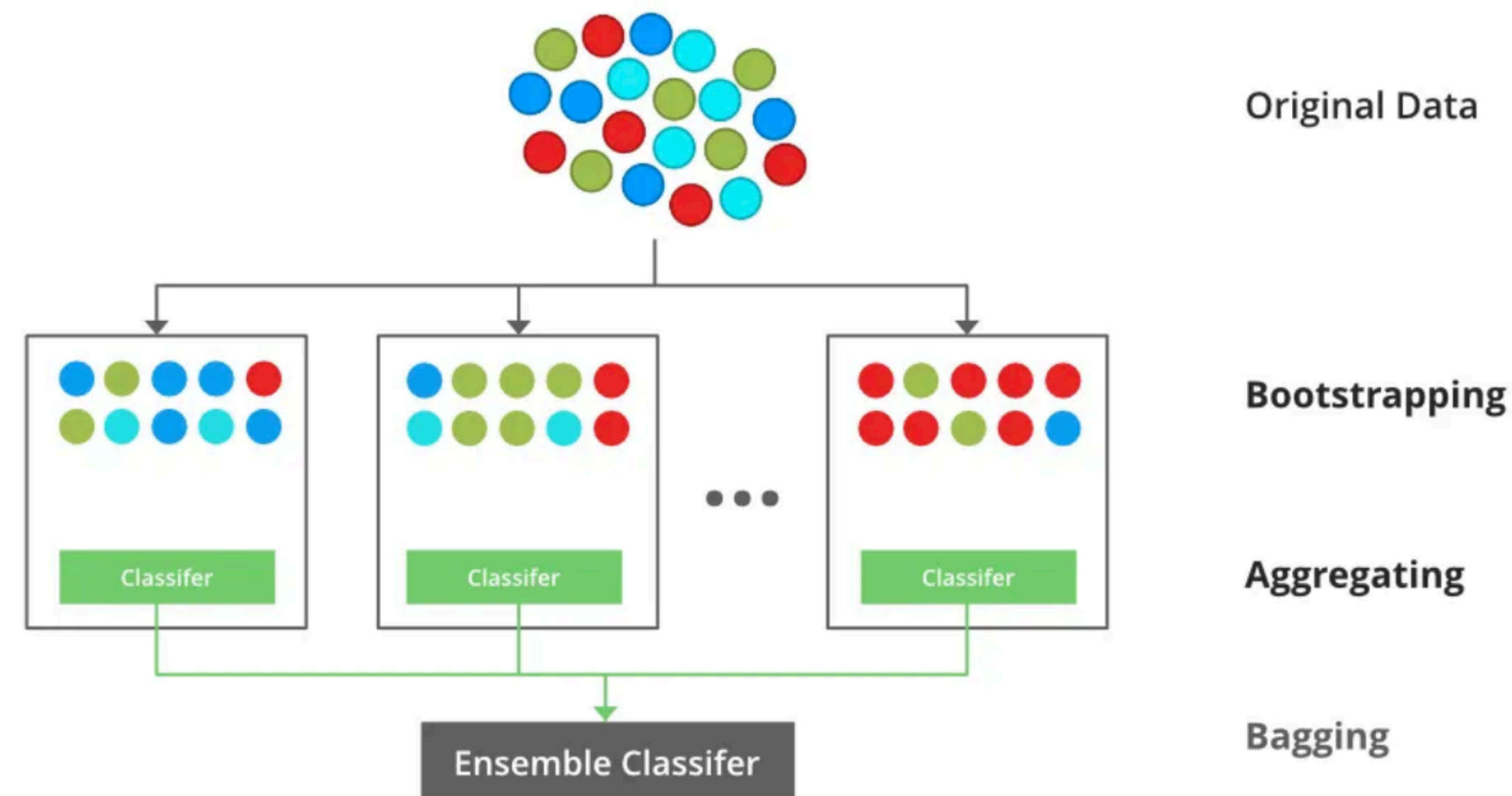
Solution: **Combine many trees** into an ensemble.

- Like asking multiple “experts” and averaging their answers.
- Results: **more stable, more accurate, less overfitting.**

# Bagging (Bootstrap Aggregating)

## Definition

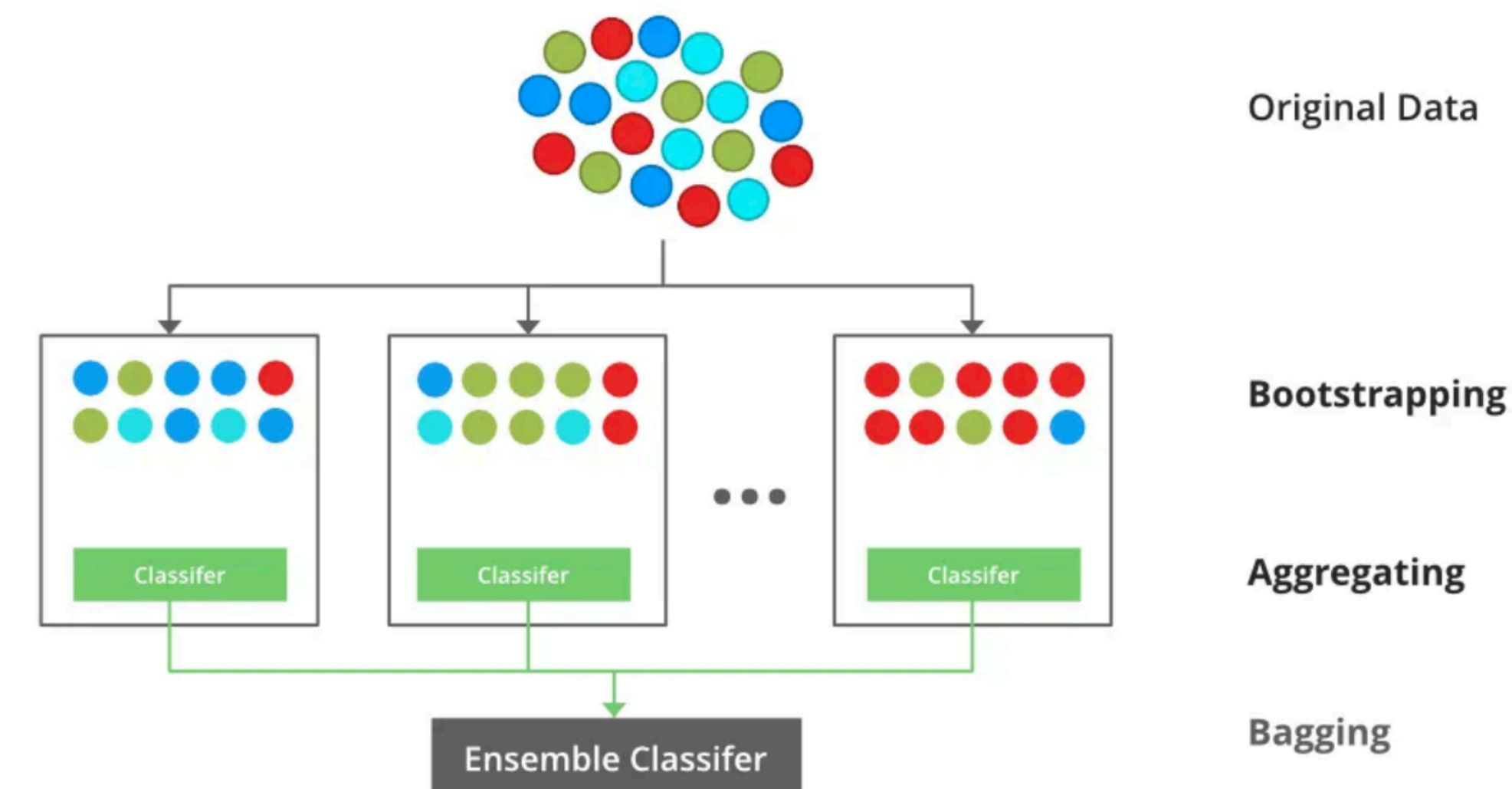
- Train many trees on different **bootstrapped samples** of the dataset.
- Each tree learns slightly different patterns.
- Final prediction = **average** (for regression) or **majority vote** (for classification).
- Reduces **variance** by averaging multiple noisy models.



# Bagging (Bootstrap Aggregating)

## Implementation Steps of Bagging:

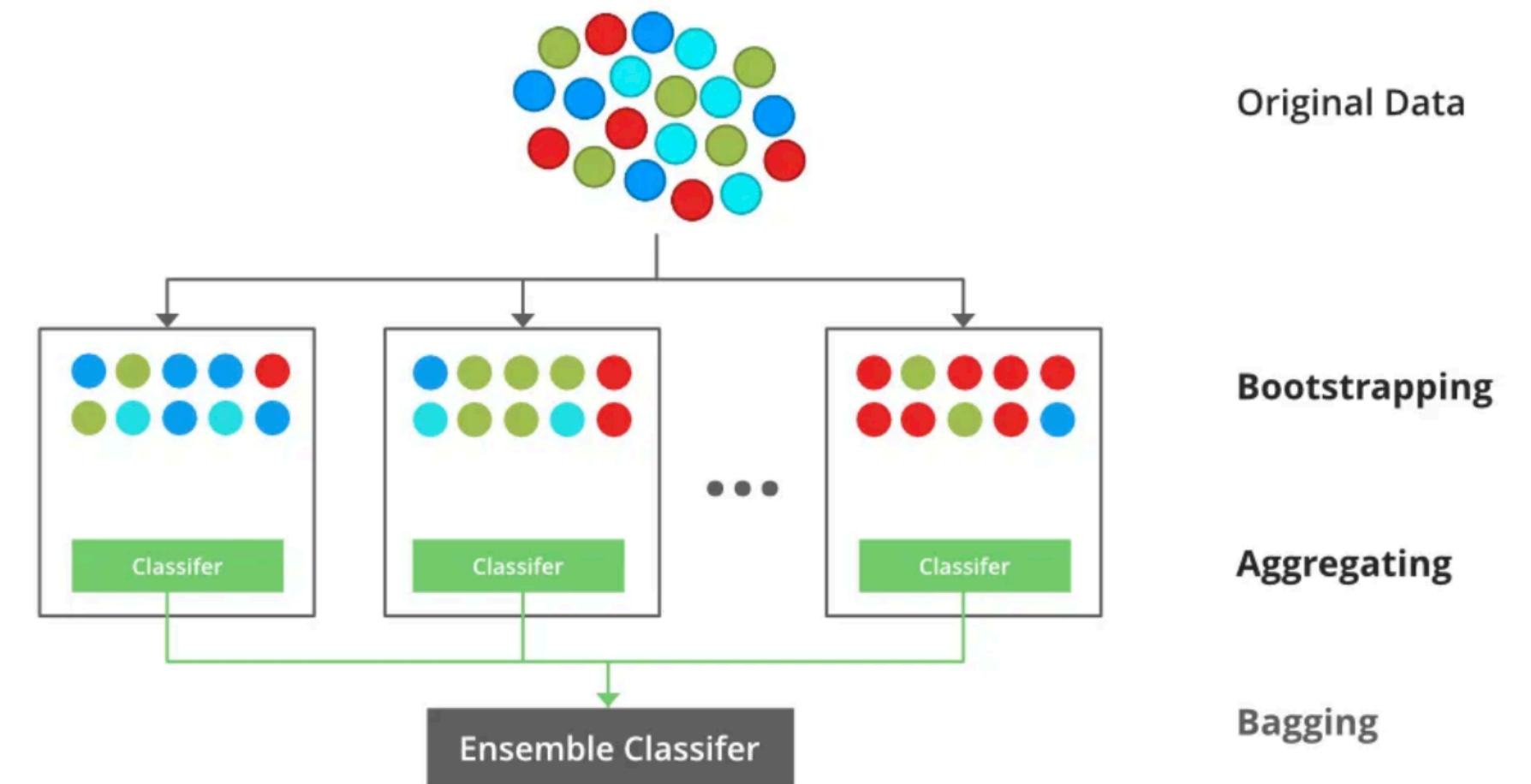
- **Step 1:** Multiple subsets are created from the original data set with equal tuples, selecting observations with replacement.
- **Step 2:** A base model is created on each of these subsets.
- **Step 3:** Each model is learned in parallel from each training set and independent of each other.
- **Step 4:** The final predictions are determined by combining the predictions from all the models.



# Bagging (Bootstrap Aggregating)

## Example:

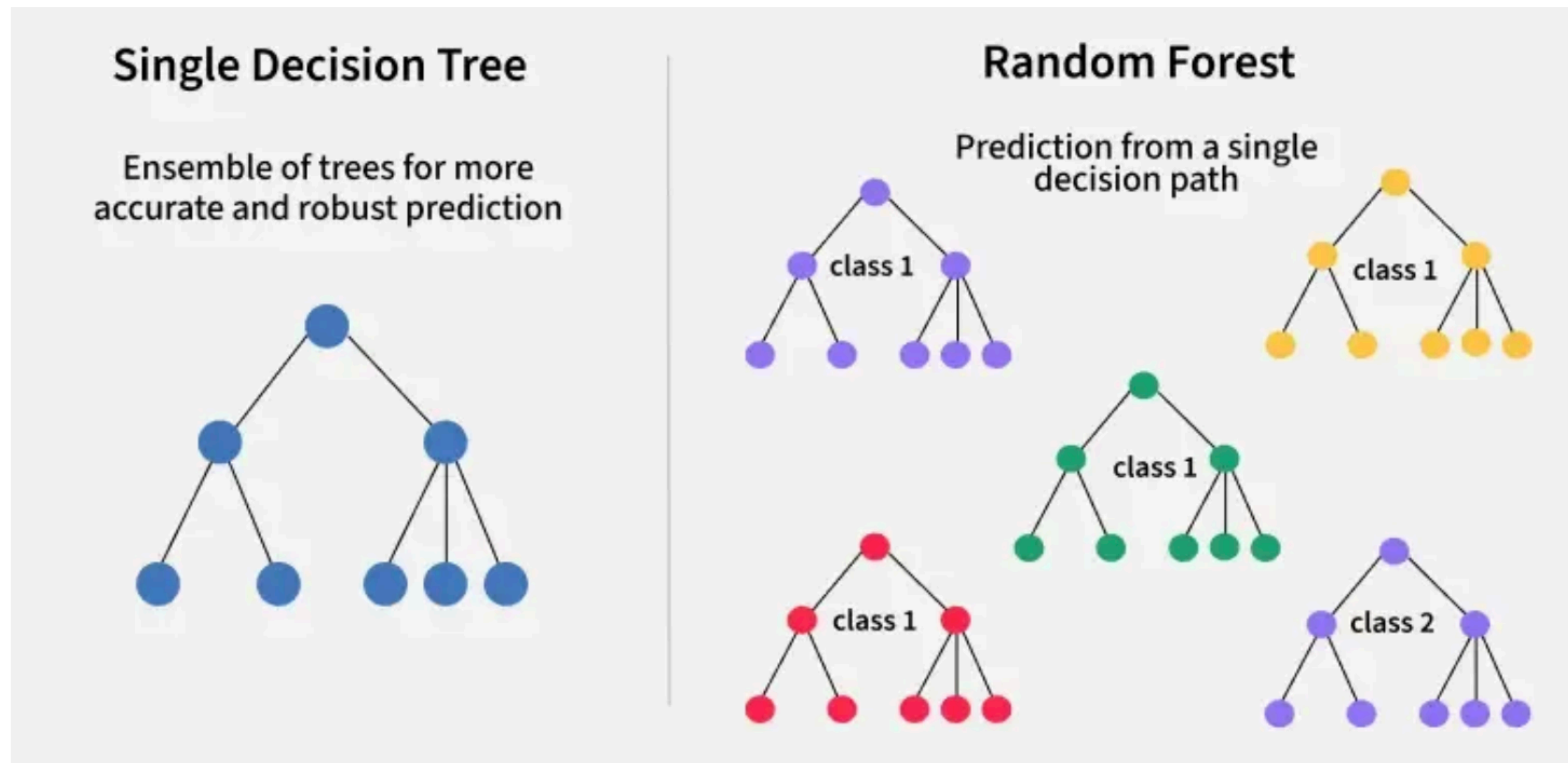
- Suppose we have 100 training samples.
- We draw 10 bootstrap samples (with replacement).
- Train 10 decision trees → each makes a prediction.
- Predictions: [120, 125, 130, 122, 128, ...]
- Bagging prediction = **average = 125** (more stable than any single tree).



# Random Forests

## Definition

- Builds on Bagging, but adds extra randomness:
- At each split, only a **random subset of features** is considered.
- This decorrelates the trees → reduces variance further.
- Works very well in practice, often default choice for tabular data.



# Random Forests

Example: Predict store sales (numeric)

## Features:

- Age, Income, Location, Weather, Ad Spending

## How Random Forest works here:

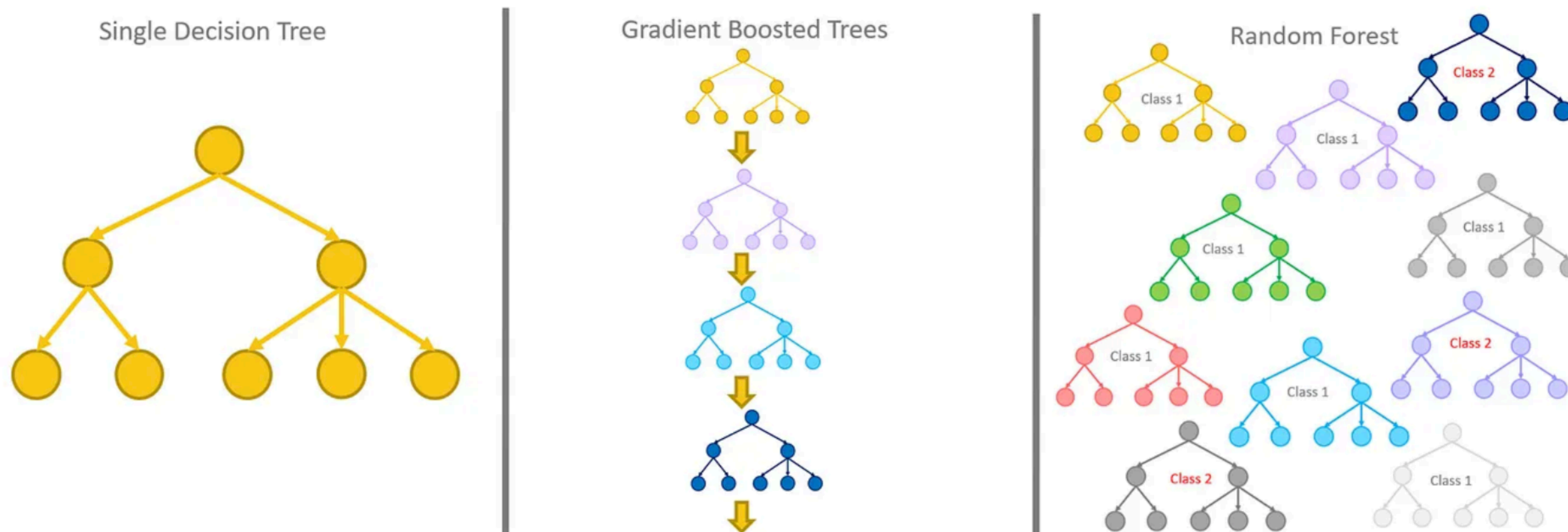
- Tree 1 might split on **Age** and **Income** to predict sales.
- Tree 2 might split on **Weather** and **Ad Spending**.
- Tree 3 might split on **Location** and **Income**.

Each tree gives a different prediction because they see **different features** and **different bootstrap samples**.

# Gradient Boosting Trees

## Definition

- Instead of training trees independently, build them **sequentially**.
- Each new tree focuses on the **errors** (residuals) of the previous trees.
- Final prediction = weighted sum of all weak learners.
- Boosting reduces **bias** (makes model more powerful).



# Gradient Boosting Trees

## Example: Predicting House Prices

- **Goal (Target):** predict house price (\$).
- **Features:** Size (sqft), Location, Age of house, #Rooms, etc.
- **Step1:** Start with a weak learner (Tree 1)
  - Train a small tree (a “stump” = 1 split).
  - Tree 1 might predict:
    - “If size > 2000 sqft → predict \$500k
    - “Else → predict \$300k”
  - It’s very rough and makes lots of mistakes.

# Gradient Boosting Trees

## Example: Predicting House Prices

- **Goal (Target):** predict house price (\$).
- **Features:** Size (sqft), Location, Age of house, #Rooms, etc.
- **Step 2: Look at errors (residuals)**
  - Train Compare actual house prices vs Tree 1 predictions.
  - Some houses priced \$700k were predicted \$500k  $\rightarrow$  residual = +\$200k.
  - Some houses priced \$250k were predicted \$300k  $\rightarrow$  residual = -\$50k.
  - These residuals = the “errors” Tree 1 failed to capture.

# Gradient Boosting Trees

## Example: Predicting House Prices

- **Goal (Target):** predict house price (\$).
- **Features:** Size (sqft), Location, Age of house, #Rooms, etc.
- **Step 3: Train Tree 2 on the residuals**
  - Train another small tree, this time trying to predict the **residuals** (the mistakes of Tree 1).
  - For example:
    - “If location = city center → add +\$200k”
    - “Else → subtract \$50k”
  - Tree 2 corrects some of Tree 1’s errors.

# Gradient Boosting Trees

## Example: Predicting House Prices

- **Goal (Target):** predict house price (\$).
- **Features:** Size (sqft), Location, Age of house, #Rooms, etc.
- **Step 4: Combine predictions**
  - New prediction = Tree 1 + Tree 2.
  - Example:
    - Tree 1 said: \$500k
    - Tree 2 said: +\$200k correction
  - Combined prediction = \$700k (much closer to true price).

# Gradient Boosting Trees

## Example: Predicting House Prices

- **Goal (Target):** predict house price (\$).
- **Features:** Size (sqft), Location, Age of house, #Rooms, etc.
- **Step 5: Repeat with more trees**
  - Train Tree 3 on what's *still wrong* (remaining residuals).
  - Add it to the model.
  - Keep going until:
    - Residuals are small, or
    - We reach a max number of trees.



# Questions?

Email: [zara.hajihashemi@sjsu.edu](mailto:zara.hajihashemi@sjsu.edu)